UNIVERSIDADE FEDERAL DE GOIÁS
INSTITUTO DE INFORMÁTICA
BACHARELADO EM ENGENHARIA DE SOFTWARE

Disciplina: **Domínios de Software** – semestre 2026/2

Atividades Práticas Supervisionadas (APS)
Projeto de Software Web para Autenticação de Usuários

Tarefa: explicação do ambiente operacional Web e projeto da arquitetura
e do desenho detalhado de componentes da aplicação

AUTORIA (ORDEM ALFABÉTICA)

Gabriel Matos Mendes

Kezer dos Santos Souza

Goiânia, 27 de outubro de 2026

SUMÁRIO

Apresentação e organização do documento **4**

Controle de autoria e revisão dos itens de informação **4**

Nota sobre o uso de inteligência artificial **5**

1 Ambiente operacional de uma aplicação Web **7**

1.1 Visão geral do ambiente operacional 7
1.2 Softwares de apoio: papel, invocação e resultados 9
1.3 Processamento de uma requisição Web de ponta a ponta 10
1.4 Processamento no navegador 13
1.5 Processamento no servidor web, no servidor de aplicação e no SGBD 14

2 Projeto da arquitetura e desenho detalhado de componentes **18**

2.1 Requisitos e escopo da solução 18
2.2 Decisões arquiteturais 19
2.3 Visões de contexto e de contêineres 20
2.4 Visão de componentes 22
2.5 Especificação da interface (API) 24
2.6 Modelo de dados 26
2.7 Desenho detalhado dos componentes 27
2.8 Comportamento dinâmico da autenticação 30
2.9 Implantação e configuração do ambiente operacional 33
2.10 Rastreabilidade entre requisitos, projeto e ambiente 34

Referências **36**

Apêndice A – Registro de comunicações e reuniões da equipe **38**

Apêndice B – Gerência de configuração no GitHub **39**

Apêndice C – Planejamento do trabalho e gestão de riscos da equipe **40**

Lista de figuras

Figura 1 – Elementos do ambiente operacional de uma aplicação Web 8
Figura 2 – Sequência de processamento de uma aplicação Web, do navegador ao SGBD 11
Figura 3 – Fluxo de processamento de uma resposta HTML no navegador 13
Figura 4 – Estrutura interna do servidor web, do servidor de aplicação e do SGBD 15
Figura 5 – Visão de contexto do Sistema Web de Autenticação (C4, nível 1) 21
Figura 6 – Visão de contêineres do Sistema Web de Autenticação (C4, nível 2) 22
Figura 7 – Visão de componentes da página de login e da API de Autenticação 23
Figura 8 – Modelo físico da tabela de usuários 26
Figura 9 – Diagrama de classes – interface da API, serviço e domínio 28
Figura 10 – Diagrama de classes – portas do núcleo e adaptadores de infraestrutura 29
Figura 11 – Sequência de autenticação entre os componentes da solução 31
Figura 12 – Diagrama de atividades da decisão de autenticação 32
Figura 13 – Diagrama de implantação 33

Lista de tabelas

Tabela 1 – Artefatos, autoria e revisão 4
Tabela 2 – Resumo da participação individual 5
Tabela 3 – Softwares de apoio do ambiente operacional 9

Tabela 4 – Classes de códigos de status HTTP e exemplos usados neste documento 12
Tabela 5 – Requisitos funcionais 18
Tabela 6 – Requisitos não funcionais 18
Tabela 7 – Decisões arquiteturais 20
Tabela 8 – Componentes, responsabilidades e local de execução 23
Tabela 9 – Contrato do recurso de autenticação 24
Tabela 10 – Campos do corpo da requisição 24
Tabela 11 – Respostas da API e software que as produz 25
Tabela 12 – Especificação das classes da API 29
Tabela 13 – Matriz de rastreabilidade 35
Tabela 14 – Registro de comunicações e reuniões 38
Tabela 15 – Estrutura do repositório 39
Tabela 16 – Registro de riscos do trabalho em equipe 40

APRESENTAÇÃO E ORGANIZAÇÃO DO DOCUMENTO

Item de informação
Autoria: Kezer dos Santos Souza Revisão: Gabriel Matos Mendes Apoio de IA: ver Nota

Este documento é o produto das Atividades Práticas Supervisionadas (APS) da disciplina Domínios de Software. Seu objetivo é projetar uma aplicação Web cujo único propósito é autenticar usuários: receber credenciais, verificar se correspondem a um usuário válido e devolver a identidade desse usuário em caso de sucesso, ou um erro de autenticação em caso de falha. O corpo do documento tem exatamente duas seções, conforme as regras da APS:

- a Seção 1 explica o ambiente operacional em que uma aplicação Web executa — navegador, sistema de nomes, pilha de rede e TLS, servidor web, servidor de aplicação e sistema gerenciador de banco de dados (SGBD) —, detalhando como cada software é invocado e que resultados produz;
- a Seção 2 apresenta o projeto da arquitetura e o desenho detalhado dos componentes da aplicação de autenticação, apoiado no ambiente descrito na Seção 1.

Cada seção é dividida em **itens de informação** numerados. No início de cada item há um bloco que identifica sua autoria e sua revisão; a Tabela 1 consolida essas informações. As referências usadas em cada item são citadas no próprio item, no sistema autor-data, e a lista completa encontra-se em Referências. Todas as citações, remissões a itens, figuras e tabelas são *links* internos, e o sumário e os marcadores do PDF permitem saltar diretamente para qualquer item. Os apêndices reúnem as evidências do trabalho em equipe: registro de comunicações (Apêndice A), gerência de configuração (Apêndice B) e planejamento e gestão de riscos (Apêndice C).

Para evitar sobreposição de conteúdo, a Seção 1 descreve o processamento Web de forma geral, usando como exemplo o carregamento da página de login, enquanto a Seção 2 trata exclusivamente da solução para o problema de autenticação. A mesma pilha tecnológica de referência — navegador, Nginx, Apache Tomcat embutido em uma aplicação Spring Boot executando na Java Virtual Machine (JVM) e PostgreSQL — é usada nas duas seções; sua escolha está justificada no item 2.2.

Controle de autoria e revisão dos itens de informação

A Tabela 1 relaciona cada item de informação (artefato) com sua autoria e sua revisão, atendendo à regra 7 da APS: cada integrante atua como autor e como revisor, e ninguém revisa um item de que é autor. A divisão foi feita por unidades lógicas — e não por fatias de texto —, de modo que o autor de um item da Seção 1 revise o item correspondente da Seção 2 que depende dele, reforçando a consistência entre as seções. A Tabela 2 resume a participação de cada integrante.

Tabela 1 – Artefatos, autoria e revisão

Item	Artefato (item de informação)	Autoria	Revisão
–	Apresentação e organização do documento	Kezer dos Santos Souza	Gabriel Matos Mendes
–	Nota sobre o uso de inteligência artificial	Gabriel Matos Mendes	Kezer dos Santos Souza
1.1	Visão geral do ambiente operacional	Gabriel Matos Mendes	Kezer dos Santos Souza
1.2	Softwares de apoio: papel, invocação e resultados	Kezer dos Santos Souza	Gabriel Matos Mendes

Item	Artefato (item de informação)	Autoria	Revisão
1.3	Processamento de uma requisição Web de ponta a ponta	Gabriel Matos Mendes	Kezer dos Santos Souza
1.4	Processamento no navegador	Kezer dos Santos Souza	Gabriel Matos Mendes
1.5	Processamento no servidor web, no servidor de aplicação e no SGBD	Gabriel Matos Mendes	Kezer dos Santos Souza
2.1	Requisitos e escopo da solução	Kezer dos Santos Souza	Gabriel Matos Mendes
2.2	Decisões arquiteturais	Gabriel Matos Mendes	Kezer dos Santos Souza
2.3	Visões de contexto e de contêineres	Kezer dos Santos Souza	Gabriel Matos Mendes
2.4	Visão de componentes	Gabriel Matos Mendes	Kezer dos Santos Souza
2.5	Especificação da interface (API)	Kezer dos Santos Souza	Gabriel Matos Mendes
2.6	Modelo de dados	Gabriel Matos Mendes	Kezer dos Santos Souza
2.7	Desenho detalhado dos componentes	Kezer dos Santos Souza	Gabriel Matos Mendes
2.8	Comportamento dinâmico da autenticação	Gabriel Matos Mendes	Kezer dos Santos Souza
2.9	Implantação e configuração do ambiente operacional	Kezer dos Santos Souza	Gabriel Matos Mendes
2.10	Rastreabilidade entre requisitos, projeto e ambiente	Gabriel Matos Mendes	Kezer dos Santos Souza
–	Apêndice A – Registro de comunicações e reuniões	Gabriel Matos Mendes	Kezer dos Santos Souza
–	Apêndice B – Gerência de configuração no GitHub	Kezer dos Santos Souza	Gabriel Matos Mendes
–	Apêndice C – Planejamento do trabalho e gestão de riscos da equipe	Gabriel Matos Mendes	Kezer dos Santos Souza

Fonte: elaborada pelos autores.

Tabela 2 – Resumo da participação individual

Integrante	Itens como autor(a)	Itens como revisor(a)
Gabriel Matos Mendes	11	9
Kezer dos Santos Souza	9	11

Fonte: elaborada pelos autores.

Nota sobre o uso de inteligência artificial

> **Item de informação**
> Autoria: Gabriel Matos Mendes Revisão: Kezer dos Santos Souza Apoio de IA: ver Nota

Em atendimento ao critério de referências da APS, que considera essencial a citação adequada do uso de inteligência artificial (IA), a equipe declara que utilizou o assistente Claude, modelo Claude Opus 5.5 (ANTHROPIC, 2026), em 27 de outubro de 2026. O assistente foi usado para: (a) produzir a versão preliminar do texto de todos os itens de informação, a partir do enunciado da APS; (b) gerar o código-fonte dos diagramas (Graphviz e SVG) e o código de montagem do PDF; e (c) sugerir as fontes bibliográficas, cujas versões vigentes foram consultadas na Web durante a elaboração (por exemplo, a publicação de julho de 2025 da norma NIST SP 800-63B-4 e os parâmetros mínimos de Argon2id recomendados pela OWASP).

A IA não substitui a responsabilidade dos integrantes: cada item tem um autor, que responde por seu conteúdo e por sua adequação ao escopo, e um revisor, que verifica correção técnica, consistência com os demais itens e conferência das referências citadas (Tabela 1). As revisões são regis-

tradas nos *pull requests* do repositório (Apêndice B). Ideias de terceiros que constam do texto são atribuídas às fontes originais, e não ao assistente, que é citado apenas como ferramenta de apoio à redação e à modelagem.

1 AMBIENTE OPERACIONAL DE UMA APLICAÇÃO WEB

Esta seção explica o que acontece, do clique do usuário até a resposta exibida na tela, quando uma aplicação Web é executada. Ela identifica os softwares de apoio — aqueles que não são desenvolvidos pela equipe, mas sem os quais a aplicação não executa —, descreve como cada um é invocado e quais resultados gera. O exemplo condutor é o acesso à página de login da aplicação projetada na Seção 2, hospedada no domínio fictício `auth.example.com`, reservado para documentação.

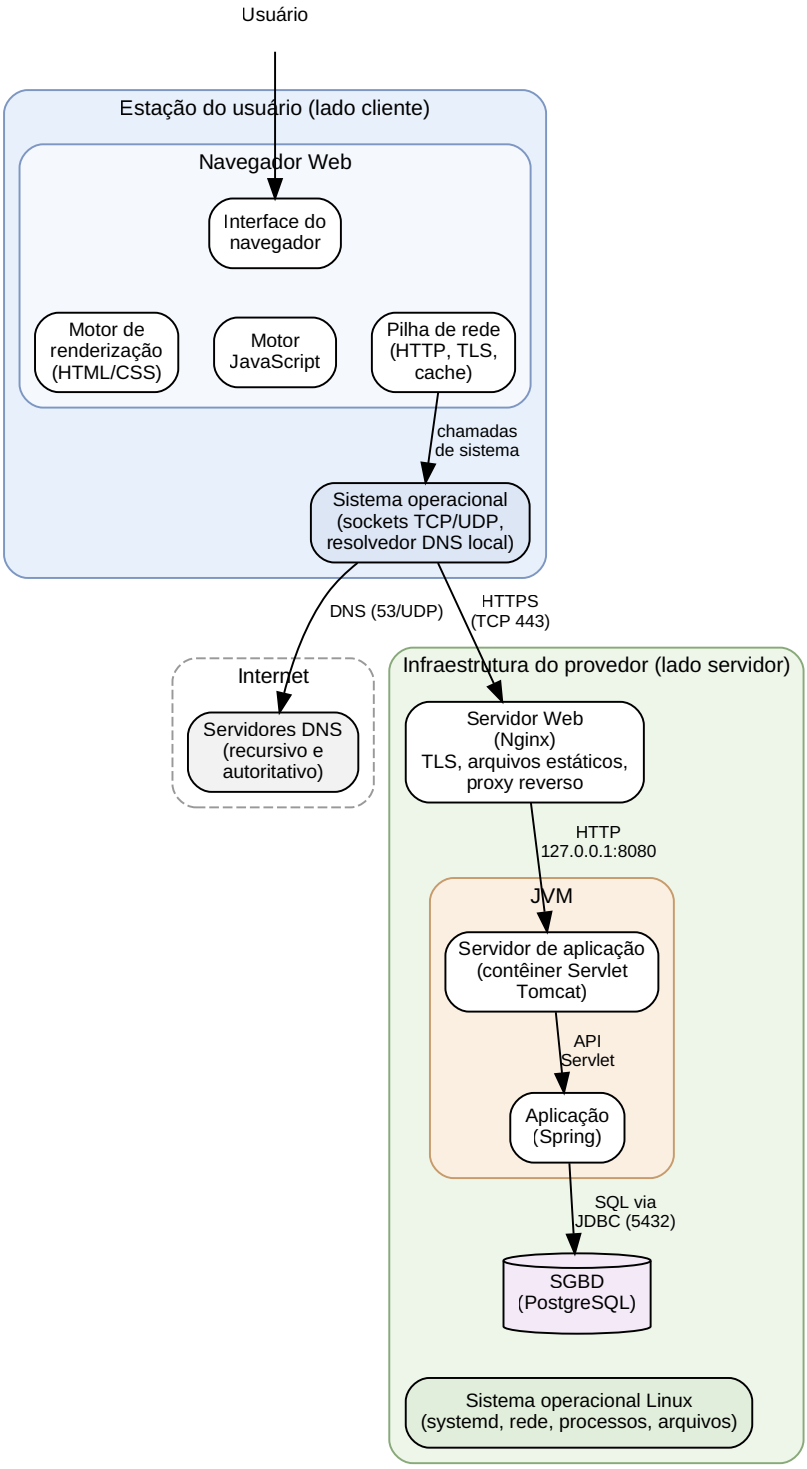
1.1 Visão geral do ambiente operacional

Item de informação 1.1 Autoria: Gabriel Matos Mendes Revisão: Kezer dos Santos Souza Apoio de IA: ver Nota

Uma aplicação Web é um software distribuído no estilo cliente-servidor: parte dela executa no navegador do usuário e parte em computadores do provedor, e as partes se comunicam pelo protocolo HTTP sobre a pilha TCP/IP da Internet (KUROSE; ROSS, 2021; FIELDING; NOTTINGHAM; RESCHKE, 2022a). O estilo arquitetural REST, que orienta a Web, acrescenta a essa organização restrições como comunicação sem estado entre requisições, interface uniforme baseada em recursos identificados por URI e possibilidade de intermediários, como *proxies* e *caches* (FIELDING, 2000).

O **ambiente operacional** é o conjunto de softwares de apoio, sistemas operacionais e recursos de rede que hospedam, invocam e transportam a aplicação. No ciclo de vida da ISO/IEC/IEEE 12207, conhecer esse ambiente é pré-requisito dos processos técnicos de definição da arquitetura, implantação (transição) e operação do software (ISO/IEC/IEEE, 2026). A Figura 1 mostra os elementos do ambiente adotado e as ligações entre eles.

Figura 1 – Elementos do ambiente operacional de uma aplicação Web



Fonte: elaborada pelos autores.

Do lado cliente, o **navegador** é o agente do usuário: interpreta HTML, CSS e JavaScript e executa a parte cliente da aplicação. Ele depende do **sistema operacional** para abrir conexões de rede e do **Sistema de Nomes de Domínio (DNS)** para converter o nome do servidor em endereço IP (MOCKAPETRIS, 1987). Do lado servidor, distinguem-se dois papéis que frequentemente são confundidos:

- o **servidor web** (aqui, Nginx) trata o protocolo HTTP na borda do sistema: aceita conexões, negocia o TLS, entrega arquivos estáticos e repassa ao servidor de aplicação as requisições que exigem processamento (NGINX, 2026);
- o **servidor de aplicação** fornece o ambiente de execução para o código da aplicação. Na plataforma Java, esse papel é desempenhado por um contêiner Jakarta Servlet — aqui, o Apache Tomcat —, que gerencia conexões, *threads* e o ciclo de vida dos componentes da aplicação (ECLIPSE FOUNDATION, 2022; APACHE SOFTWARE FOUNDATION, 2026).

Completam o ambiente a **máquina virtual Java**, que executa o código compilado; o *framework* **Spring**, que organiza a aplicação dentro do contêiner (SPRING, 2026); e o **SGBD PostgreSQL**, que armazena os dados persistentes (POSTGRESQL GLOBAL DEVELOPMENT GROUP, 2026). O item 1.2 detalha cada um desses softwares, e os item 1.3 a item 1.5 descrevem o processamento que eles realizam em conjunto.

1.2 Softwares de apoio: papel, invocação e resultados

Item de informação 1.2 Autoria: Kezer dos Santos Souza Revisão: Gabriel Matos Mendes Apoio de IA: ver Nota

A Tabela 3 descreve, para cada software de apoio, seu papel, a forma como é iniciado (quando passa a existir como processo), a forma como é invocado durante o processamento de uma requisição e os resultados que produz. A distinção entre *iniciar* e *invocar* é importante: os servidores são iniciados uma vez, normalmente pelo gerenciador de serviços do sistema operacional (`systemd`), e permanecem à espera de eventos de rede; cada requisição que chega é uma nova invocação de seu código de tratamento (SOMMERVILLE, 2019).

Tabela 3 – Softwares de apoio do ambiente operacional

Software	Papel	Como é iniciado	Como é invocado em execução	Resultados gerados
Navegador Web (ex.: Firefox, Chrome)	Agente do usuário: obtém recursos, interpreta HTML/CSS e executa JavaScript.	Pelo usuário; o sistema operacional cria o processo principal e processos auxiliares (renderização, rede, GPU).	Quando o usuário digita uma URL, segue um *link* ou envia um formulário, ou quando um *script* chama `fetch()`.	Requisições HTTP; página renderizada (DOM) na tela; entradas de *cache*; mensagens no console em caso de erro.
Resolvedor DNS (local e recursivo)	Traduz nomes de domínio em endereços IP.	Serviço do sistema operacional (ex.: `systemd-resolved`) e servidores recursivos do provedor de acesso.	Pelo navegador, via chamada `getaddrinfo()`, quando o nome não está em *cache*.	Registros A/AAAA com prazo de validade (TTL) ou erro (ex.: NXDOMAIN).
Sistema operacional (cliente e servidor)	Gerencia processos, memória, arquivos e a pilha TCP/IP.	Na inicialização da máquina.	Por chamadas de sistema: `socket`, `connect`, `accept`, `read`, `write`, `epoll_wait`.	Conexões TCP estabelecidas, descritores de arquivo, eventos de prontidão de E/S.

Software	Papel	Como é iniciado	Como é invocado em execução	Resultados gerados
Biblioteca TLS (no navegador e OpenSSL no servidor)	Autentica o servidor por certificado X.509 e cifra o canal.	Carregada como biblioteca pelo navegador e pelo Nginx.	No *handshake* de cada nova conexão e em cada leitura/escrita cifrada.	Canal cifrado com chaves de sessão; alerta de erro se o certificado for inválido.
Servidor web Nginx	Porta de entrada HTTP(S): termina o TLS, entrega estáticos, atua como *proxy* reverso, limita taxa e registra acessos.	`systemctl start nginx`: o processo mestre lê `nginx.conf`, abre as portas 80 e 443 e cria processos trabalhadores; `nginx -s reload` recarrega a configuração.	Por evento de rede: o trabalhador é notificado pelo sistema operacional (`epoll`) quando chega uma conexão ou dados.	Respostas HTTP (arquivo ou resposta do servidor de aplicação); códigos de *status* próprios (301, 404, 413, 429, 502, 504); linhas em `access.log` e `error.log`.
JVM	Executa o *bytecode* Java, com compilação JIT e coleta de lixo.	`java -jar autenticacao.jar`, executado por uma unidade do `systemd`.	Uma única vez; a partir daí executa continuamente as *threads* da aplicação.	Processo da aplicação em execução; registros de coleta de lixo e erros fatais.
Servidor de aplicação (Tomcat embutido)	Contêiner Servlet: aceita requisições do Nginx, aloca *threads*, aplica filtros e invoca servlets.	Pelo Spring Boot durante a inicialização da JVM: cria o conector HTTP em 127.0.0.1:8080 e registra o `DispatcherServlet`.	A cada requisição: o conector a aceita, uma *thread* do *pool* chama `Filter.doFilter()` e `Servlet.service()`.	Objeto de resposta HTTP (status, cabeçalhos, corpo JSON); registros da aplicação.
Spring Framework / Spring Boot	Injeção de dependências, mapeamento de requisições, conversão JSON, validação e acesso a dados.	Na inicialização, ao criar o contexto da aplicação (`ApplicationContext`).	Pelo `DispatcherServlet`, que localiza e chama o método controlador adequado.	Chamada do código da aplicação; objetos convertidos de/para JSON; respostas de erro padronizadas.
Driver JDBC e *pool* de conexões (PostgreSQL JDBC, HikariCP)	Abrem e reutilizam conexões com o SGBD.	Na inicialização do contexto, que abre as conexões mínimas do *pool*.	Quando um componente de acesso a dados executa uma consulta.	Conexão emprestada; conjunto de linhas (`ResultSet`); exceções de acesso a dados.
SGBD PostgreSQL	Armazena e consulta os dados persistentes.	`systemctl start postgresql`: o processo `postmaster` passa a escutar a porta 5432.	A cada nova conexão, cria um processo *backend* dedicado, que atende as consultas dessa conexão.	Linhas resultantes, erros SQL e registros no *log* do servidor.

Fonte: elaborada pelos autores.

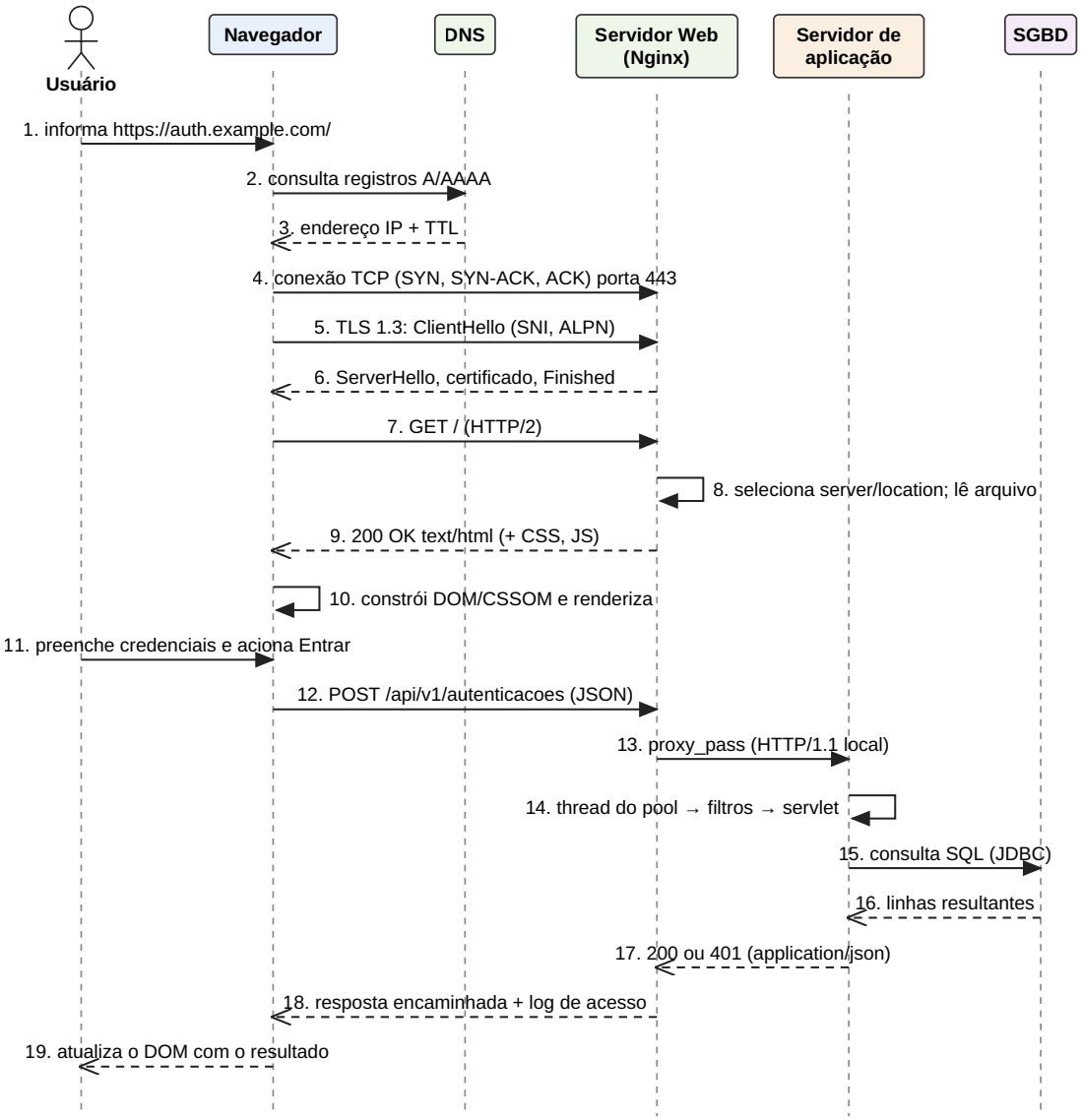
Todos os componentes do lado servidor são processos de longa duração: uma falha em qualquer um deles produz um resultado observável para o cliente. Por exemplo, se a JVM não estiver em execução, o Nginx não consegue conectar-se ao servidor de aplicação e responde `502 Bad Gateway`; se o servidor de aplicação demorar além do limite configurado, a resposta é `504 Gateway Timeout` (NGINX, 2026; FIELDING; NOTTINGHAM; RESCHKE, 2022a).

1.3 Processamento de uma requisição Web de ponta a ponta

Item de informação 1.3 Autoria: Gabriel Matos Mendes Revisão: Kezer dos Santos Souza Apoio de IA: ver Nota

A Figura 2 mostra, em um diagrama de sequência UML (OMG, 2017), a colaboração entre os softwares de apoio desde o momento em que o usuário informa o endereço da aplicação até a atualização da tela com o resultado de uma requisição ao servidor de aplicação. Os números das mensagens são usados no texto a seguir.

Figura 2 – Sequência de processamento de uma aplicação Web, do navegador ao SGBD



Fonte: elaborada pelos autores.

a) Resolução de nome (mensagens 1 a 3)

O navegador decompõe a URL `https://auth.example.com/` em esquema (`https`), nome do servidor (`auth.example.com`), porta implícita (443) e caminho (`/`). Se o endereço IP do nome não estiver em *cache*, o navegador solicita ao sistema operacional a resolução, que consulta o servidor DNS recursivo; este percorre a hierarquia de servidores autoritativos e devolve os registros A (IPv4) ou AAAA (IPv6), com o prazo durante o qual a resposta pode ser reutilizada (MOCKAPETRIS, 1987; KUROSE; ROSS, 2021).

b) Conexão e segurança do canal (mensagens 4 a 6)

Com o endereço IP, o sistema operacional estabelece uma conexão TCP com a porta 443 do servidor pelo *handshake* de três vias. Em seguida, a biblioteca TLS do navegador inicia o *handshake* TLS 1.3: a mensagem *ClientHello* informa o nome do servidor desejado (SNI) e os protocolos de aplicação suportados (ALPN, por exemplo `h2` para HTTP/2). O Nginx responde com o certificado do domínio, que o navegador valida contra as autoridades certificadoras confiáveis; ao final, ambos derivam chaves simétricas, e todo o tráfego seguinte é cifrado e protegido contra alteração (RESCORLA, 2018).

c) Requisição HTTP (mensagem 7)

O navegador envia uma mensagem de requisição composta por método, alvo, cabeçalhos e, opcionalmente, corpo. A semântica dessas mensagens é a mesma em todas as versões do protocolo (FIELDING; NOTTINGHAM; RESCHKE, 2022a); o que muda é o enquadramento: em HTTP/1.1 a mensagem é texto (FIELDING; NOTTINGHAM; RESCHKE, 2022b), enquanto em HTTP/2 é dividida em quadros binários multiplexados na mesma conexão (THOMSON; BENFIELD, 2022). A forma textual, mais legível, é:

```
GET / HTTP/1.1
Host: auth.example.com
User-Agent: Mozilla/5.0 (X11; Linux x86_64) Firefox/…
Accept: text/html,application/xhtml+xml
Accept-Encoding: gzip, br
Accept-Language: pt-BR,pt;q=0.9
```

d) Processamento no servidor e resposta (mensagens 8 e 9)

O Nginx seleciona o bloco de configuração correspondente ao nome `auth.example.com` e ao caminho `/`; como a página de login é um arquivo estático, ele o lê do disco e devolve a resposta sem envolver o servidor de aplicação (detalhes no item 1.5). A resposta contém linha de *status*, cabeçalhos e corpo:

```
HTTP/1.1 200 OK
Server: nginx
Date: Tue, 27 Oct 2026 18:30:00 GMT
Content-Type: text/html; charset=utf-8
Content-Length: 1534
Strict-Transport-Security: max-age=31536000; includeSubDomains
Content-Security-Policy: default-src 'self'; frame-ancestors 'none'

<!doctype html><html lang="pt-BR"> … </html>
```

O código de *status* é o principal resultado da invocação do servidor, e sua classe indica como o cliente deve proceder, como mostra a Tabela 4. O cabeçalho `Strict-Transport-Security` instrui o navegador a usar somente HTTPS nesse domínio durante o prazo indicado (HODGES; JACKSON; BARTH, 2012).

Tabela 4 – Classes de códigos de status HTTP e exemplos usados neste documento

Classe	Significado	Exemplos relevantes para a aplicação
1xx	Informativa: processamento em andamento.	Não utilizados pela aplicação.

Classe	Significado	Exemplos relevantes para a aplicação
2xx	Sucesso: a requisição foi recebida, entendida e aceita.	`200 OK` – página entregue ou usuário autenticado.
3xx	Redirecionamento: o cliente deve agir em outro local.	`301 Moved Permanently` – de `http://` para `https://`.
4xx	Erro do cliente: a requisição tem problema ou não pode ser atendida.	`400` dados inválidos; `401` não autenticado; `404` recurso inexistente; `413` corpo grande demais; `429` excesso de requisições.
5xx	Erro do servidor: a requisição era válida, mas o servidor falhou.	`500` erro interno; `502` servidor de aplicação inacessível; `503` serviço indisponível; `504` tempo esgotado.

Fonte: elaborada pelos autores.

e) Renderização e interação (mensagens 10 a 19)

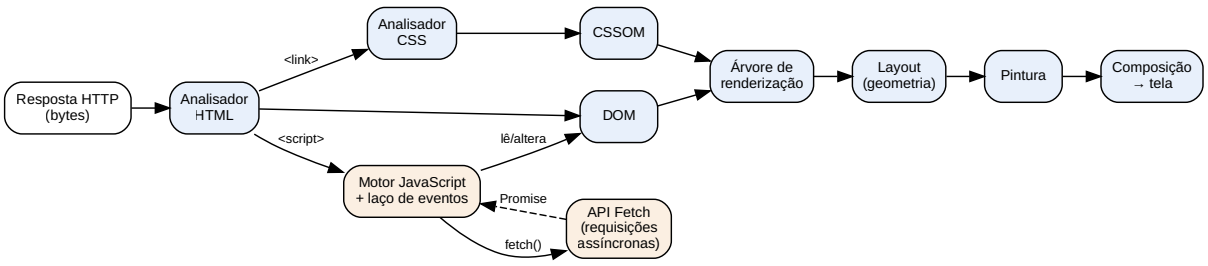
Ao receber o HTML, o navegador constrói e exibe a página, buscando com novas requisições os recursos referenciados, como folhas de estilo e *scripts* (item 1.4). Quando o usuário aciona o botão de envio, o código JavaScript da página envia ao mesmo servidor uma requisição `POST` com corpo JSON. Dessa vez o Nginx não encontra um arquivo a entregar: a configuração determina que caminhos iniciados por `/api/` sejam repassados ao servidor de aplicação, que executa o código da aplicação, consulta o SGBD e produz a resposta. O Nginx devolve essa resposta ao navegador, registra o acesso, e o JavaScript atualiza a página sem recarregá-la. O contrato exato dessa requisição pertence à solução de autenticação e está especificado no item 2.5.

1.4 Processamento no navegador

Item de informação 1.4 Autoria: Kezer dos Santos Souza Revisão: Gabriel Matos Mendes Apoio de IA: ver Nota

O navegador é, ao mesmo tempo, cliente HTTP e ambiente de execução da parte cliente da aplicação. Seu processamento é descrito pelas especificações HTML e Fetch, mantidas pelo WHATWG (WHATWG, 2026a; WHATWG, 2026b), e está resumido na Figura 3.

Figura 3 – Fluxo de processamento de uma resposta HTML no navegador



Fonte: elaborada pelos autores com base em Grigorik (2013) e WHATWG (2026a).

Construção da página. Os *bytes* recebidos são decodificados segundo o `charset` do cabeçalho `Content-Type` e entregues ao analisador HTML, que produz incrementalmente o DOM (*Document Object Model*), a árvore de objetos que representa o documento. Ao encontrar `<link rel="stylesheet">` ou `<script src>`, o navegador dispara novas requisições, reutilizando a conexão já aberta. O CSS é transformado no CSSOM; DOM e CSSOM são combinados na árvore

de renderização, que contém apenas os elementos visíveis com seus estilos calculados. Seguem as etapas de *layout* (cálculo de posição e tamanho de cada caixa), pintura (geração dos *pixels*) e composição das camadas na tela (GRIGORIK, 2013).

Execução de JavaScript. O motor JavaScript executa os *scripts* da página em uma única *thread* principal, regida pelo laço de eventos (*event loop*): eventos do usuário, como o envio de um formulário, e conclusões de operações de rede são enfileirados como tarefas e processados um de cada vez. Um tratador de eventos pode ler e alterar o DOM; cada alteração provoca novo cálculo de estilo, *layout* e pintura da parte afetada (WHATWG, 2026a).

Requisições assíncronas. A função `fetch()` envia requisições HTTP sem recarregar a página e devolve uma *Promise*, resolvida quando a resposta chega; o código decide então o que exibir conforme o código de *status* e o corpo recebido (WHATWG, 2026b). Esse é o mecanismo usado pela página de login para enviar as credenciais e tratar o resultado (item 2.4).

Políticas de segurança aplicadas pelo navegador. A política de mesma origem (*same-origin policy*) impede que um *script* leia respostas de origens diferentes da sua, salvo autorização explícita do servidor pelo mecanismo CORS. Como a página e a API de autenticação são servidas pela mesma origem, `https://auth.example.com`, não é necessário habilitar CORS — uma decisão registrada no item 2.2. O navegador também aplica a política de conteúdo (`Content-Security-Policy`), que restringe de onde podem ser carregados *scripts* e estilos, e a política HSTS recebida na resposta (WHATWG, 2026b; HODGES; JACKSON; BARTH, 2012).

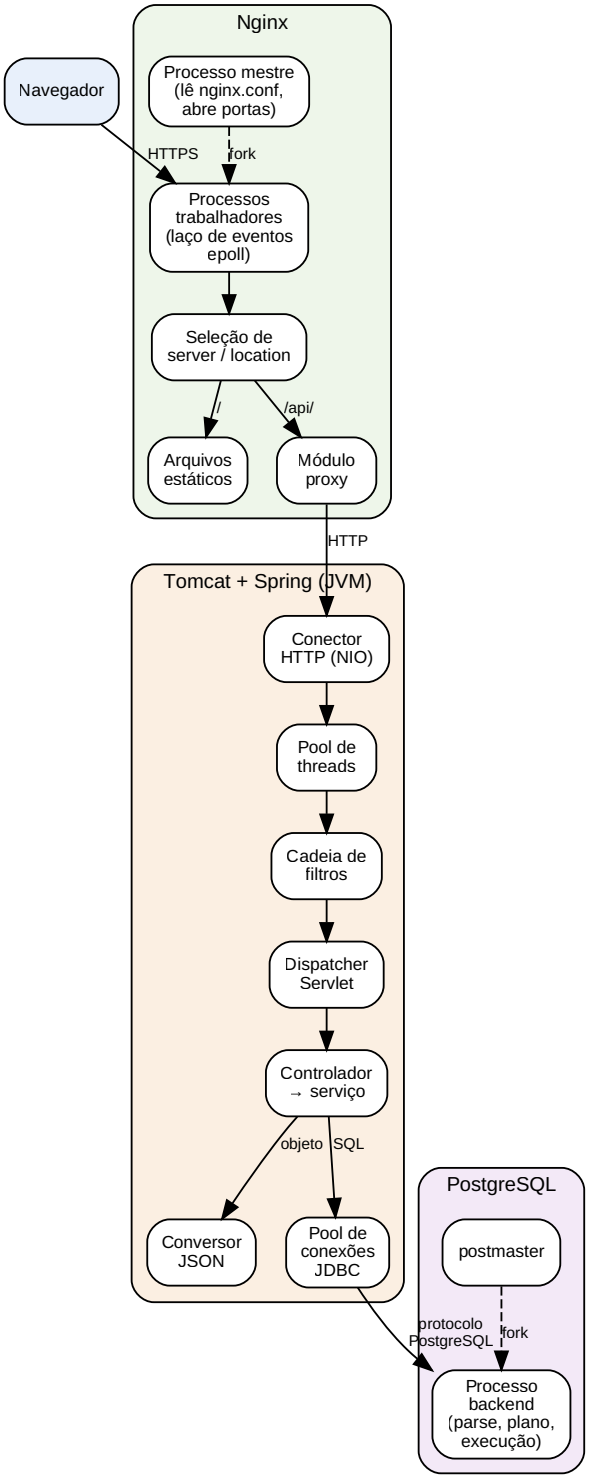
Resultados. O processamento no navegador produz a página exibida ao usuário, novas requisições HTTP (para recursos e para a API), alterações no DOM em resposta às interações e, em caso de falhas, mensagens no console de desenvolvimento e páginas de erro do próprio navegador, como a de certificado inválido.

1.5 Processamento no servidor web, no servidor de aplicação e no SGBD

Item de informação 1.5 Autoria: Gabriel Matos Mendes Revisão: Kezer dos Santos Souza Apoio de IA: ver Nota

A Figura 4 detalha a estrutura interna dos três softwares do lado servidor e o caminho percorrido por uma requisição dirigida à API. A configuração concreta desses softwares para a aplicação de autenticação é apresentada no item 2.9.

Figura 4 – Estrutura interna do servidor web, do servidor de aplicação e do SGBD



Fonte: elaborada pelos autores com base em NGINX (2026), Apache Software Foundation (2026), SPRING (2026) e PostgreSQL Global Development Group (2026).

a) Servidor web (Nginx)

O Nginx adota uma arquitetura orientada a eventos. O processo mestre, executado com privilégios administrativos, lê a configuração, abre as portas e cria processos trabalhadores sem privilégios, normalmente um por núcleo de processador. Cada trabalhador executa um laço de eventos não

bloqueante que atende milhares de conexões simultâneas: em vez de dedicar uma *thread* a cada conexão, ele pede ao sistema operacional (via `epoll`) que o avise quando há dados a ler ou espaço para escrever (NGINX, 2026). Para cada requisição, o trabalhador:

1. conclui o *handshake* TLS e lê a linha de requisição e os cabeçalhos;
2. escolhe o bloco `server` pelo nome do servidor (SNI/`Host`) e, dentro dele, o bloco `location` que melhor corresponde ao caminho;
3. executa as fases de processamento configuradas, como limitação de taxa e restrição de tamanho do corpo, que podem encerrar a requisição com `429` ou `413`;
4. gera o conteúdo: lê um arquivo estático e o envia, ou, no caso de um `location` com `proxy_pass`, abre uma conexão HTTP com o servidor de aplicação, repassa a requisição acrescentando os cabeçalhos `X-Forwarded-For` e `X-Forwarded-Proto` (endereço do cliente e protocolo original) e aguarda a resposta dentro dos limites de tempo configurados;
5. devolve a resposta ao cliente e grava uma linha no *log* de acesso com endereço do cliente, data, requisição, *status*, tamanho e tempo de resposta.

b) Servidor de aplicação (Tomcat com Spring)

No Tomcat, o **conector HTTP** aceita a conexão vinda do Nginx e, com um seletor não bloqueante, detecta quando há uma requisição completa; nesse momento entrega o processamento a uma *thread* de um *pool* de tamanho limitado, que atenderá essa requisição do início ao fim. A *thread* analisa a mensagem, cria os objetos `HttpServletRequest` e `HttpServletResponse` e os conduz pela cadeia de filtros (`Filter`) até o servlet mapeado para o caminho (APACHE SOFTWARE FOUNDATION, 2026). O contêiner é responsável pelo ciclo de vida dos servlets — criação e inicialização (`init`), atendimento de requisições (`service`) e destruição (`destroy`) — e pela execução concorrente de requisições (ECLIPSE FOUNDATION, 2022).

Na aplicação Spring, há um único servlet, o `DispatcherServlet`, que implementa o padrão *Front Controller* (FOWLER, 2002): ele consulta os mapeamentos para descobrir qual método controlador atende ao método HTTP e ao caminho recebidos, converte o corpo JSON em objeto Java por meio de um conversor de mensagens, aplica as validações declaradas, invoca o controlador e converte o valor devolvido de volta para JSON. Exceções lançadas pelo código da aplicação são capturadas por tratadores de exceção, que as transformam em respostas de erro (SPRING, 2026).

c) Acesso ao SGBD (PostgreSQL)

Quando o código da aplicação precisa de dados, pede uma conexão ao *pool*, que entrega uma conexão já aberta e autenticada, evitando o custo de estabelecer uma nova a cada requisição. O driver JDBC envia a consulta parametrizada pelo protocolo do PostgreSQL. No servidor, o processo `postmaster` criou, no momento da conexão, um processo *backend* dedicado, que agora analisa o comando SQL, reescreve-o, escolhe um plano de execução — por exemplo, a busca por um índice — e o executa, devolvendo as linhas encontradas (POSTGRESQL GLOBAL DEVELOPMENT GROUP, 2026). Terminada a consulta, a conexão volta ao *pool*, e a resposta percorre o caminho inverso: controlador, conversor JSON, Tomcat, Nginx e navegador.

Os resultados observáveis desse processamento são a resposta HTTP entregue ao cliente e os registros de cada camada — *log* de acesso do Nginx, *log* da aplicação e *log* do SGBD —, que permitem reconstruir o que ocorreu com cada requisição.

2 PROJETO DA ARQUITETURA E DESENHO DETALHADO DE COMPONENTES

Esta seção corresponde aos processos técnicos de definição da arquitetura e de definição do desenho do ciclo de vida da ISO/IEC/IEEE 12207 (ISO/IEC/IEEE, 2026). A arquitetura é descrita por um conjunto de visões complementares, porque nenhuma estrutura isolada representa todas as propriedades relevantes de um sistema (BASS; CLEMENTS; KAZMAN, 2021): visões de contexto, contêineres e componentes, segundo o modelo C4 (BROWN, 2026), e diagramas UML de classes, sequência, atividades e implantação para o desenho detalhado (OMG, 2017). O ponto de partida são os requisitos derivados do escopo da APS (item 2.1) e o ponto de chegada é a rastreabilidade entre requisitos, elementos de projeto e ambiente operacional (item 2.10).

2.1 Requisitos e escopo da solução

Item de informação 2.1 Autoria: Kezer dos Santos Souza Revisão: Gabriel Matos Mendes Apoio de IA: ver Nota

O escopo definido no Apêndice A do enunciado da APS restringe a aplicação à verificação de credenciais: ela recebe as credenciais, verifica a existência do usuário, compara a credencial fornecida com a registrada e decide se o usuário está ou não autenticado. Os requisitos funcionais derivados desse escopo estão na Tabela 5; os requisitos não funcionais, classificados segundo o modelo de qualidade da ISO/IEC 25010 (ISO/IEC, 2023), estão na Tabela 6.

Tabela 5 – Requisitos funcionais

Id.	Requisito	Origem no escopo
RF01	Receber credenciais enviadas pelo cliente, compostas por um identificador (nome de usuário ou e-mail) e uma senha.	Entrada de credenciais
RF02	Rejeitar requisições com credenciais ausentes, vazias ou fora dos limites de tamanho, sem consultar o repositório.	Entrada de credenciais
RF03	Verificar a existência do usuário no repositório de dados, buscando pelo nome de usuário ou pelo e-mail, sem distinção entre maiúsculas e minúsculas.	Validação das credenciais
RF04	Comparar a senha fornecida com a credencial registrada, que é um *hash* de senha.	Comparação da credencial
RF05	Decidir entre *autenticado* (usuário existe e senha confere) e *não autenticado* (usuário inexistente ou senha incorreta).	Decisão de autenticação
RF06	Em caso de sucesso, retornar um objeto com a identificação do usuário (identificador único, nome de usuário, e-mail, nome de exibição e instante da autenticação).	Retorno em caso de sucesso
RF07	Em caso de falha, retornar um erro de autenticação que não revele qual das condições falhou.	Retorno em caso de falha

Fonte: elaborada pelos autores.

Tabela 6 – Requisitos não funcionais

Id.	Característica (ISO/IEC 25010)	Requisito
RNF01	Segurança – confidencialidade	Credenciais trafegam somente por HTTPS (TLS 1.2 ou 1.3); requisições HTTP são redirecionadas e o navegador é instruído por HSTS a não usar HTTP (RESCORLA, 2018; HODGES; JACKSON; BARTH, 2012).

Id.	Característica (ISO/IEC 25010)	Requisito
RNF02	Segurança – confidencialidade	Senhas nunca são armazenadas nem comparadas em claro: o repositório guarda *hashes* Argon2id com sal único por usuário e parâmetros mínimos de 19 MiB de memória, 2 iterações e paralelismo 1 (OWASP, 2026b; BIRYUKOV *et al.*, 2021).
RNF03	Segurança – confidencialidade	Não enumeração de usuários: usuário inexistente e senha incorreta produzem a mesma resposta (status e corpo) e tempo de processamento semelhante (OWASP, 2026a).
RNF04	Segurança – resistência	Limitação da taxa de tentativas por endereço IP, para conter ataques de força bruta e adivinhação em massa (TEMOSHOK *et al.*, 2025).
RNF05	Segurança – responsabilização	Cada tentativa gera um registro de evento com resultado e identificador mascarado; a senha nunca aparece em registros ou mensagens de erro (OWASP, 2026a).
RNF06	Segurança – integridade	Privilégio mínimo: a aplicação acessa o banco com papel somente de leitura; servidor de aplicação e SGBD não são acessíveis diretamente pela Internet.
RNF07	Eficiência de desempenho	Meta de projeto: 95% das autenticações respondidas em até 1 s com carga de 20 autenticações por segundo em um nó, com consumo de memória limitado.
RNF08	Flexibilidade – escalabilidade	A API não mantém estado entre requisições, permitindo replicar instâncias atrás do servidor web.
RNF09	Usabilidade e compatibilidade	Aceitar senhas com caracteres Unicode, normalizadas antes da verificação, permitir colar a senha e uso de gerenciadores de senhas (TEMOSHOK *et al.*, 2025).
RNF10	Manutenibilidade – modularidade	As regras de autenticação independem de *framework*, servidor e SGBD, que são substituíveis por adaptadores (MARTIN, 2019).

Fonte: elaborada pelos autores.

Fora do escopo. Por determinação do enunciado (“somente verificação de credenciais, nada além disso”), não fazem parte da solução: cadastro de usuários e definição de senhas, alteração e recuperação de senha, emissão de sessões, *cookies* ou *tokens*, autorização (controle de acesso a recursos), autenticação multifator, federação de identidade e bloqueio de contas. O repositório de usuários é considerado previamente populado por outro processo, que grava os *hashes* no formato definido no item 2.6.

> **Limitação reconhecida.** A norma NIST SP 800-63B-4 exige que o verificador limite o número de tentativas malsucedidas por conta (TEMOSHOK *et al.*, 2025). Esse controle requer manter contadores por usuário, o que implica gravar estado e ultrapassa o escopo de “somente verificação”. Na solução, a limitação é feita por endereço IP na borda (RNF04, item 2.9); a limitação por conta é recomendada como evolução, em componente separado.

2.2 Decisões arquiteturais

Item de informação 2.2 Autoria: Gabriel Matos Mendes Revisão: Kezer dos Santos Souza Apoio de IA: ver Nota

A Tabela 7 registra as principais decisões arquiteturais, as alternativas consideradas e os requisitos que motivaram cada escolha. O registro explícito das decisões e de suas justificativas facilita a avaliação da arquitetura e sua evolução (BASS; CLEMENTS; KAZMAN, 2021; PRESSMAN; MAXIM, 2021).

Tabela 7 – Decisões arquiteturais

Id.	Decisão	Alternativas consideradas	Justificativa	Requisitos
DA01	Cliente-servidor com página estática e API REST/JSON (`POST /api/v1/autenticacoes`).	Formulário HTML tradicional com página gerada no servidor.	Separa apresentação e verificação; a API pode ser usada por outros clientes; segue as restrições REST de interface uniforme e ausência de estado (FIELDING, 2000).	RF01, RF06, RNF08
DA02	Página e API servidas pela mesma origem, atrás do Nginx.	API em subdomínio próprio com CORS.	Dispensa CORS, reduz a superfície de ataque e simplifica a política de conteúdo (item 1.4).	RNF01
DA03	Nginx como servidor web e *proxy* reverso; Tomcat embutido no Spring Boot como servidor de aplicação.	Apache httpd; servidor Jakarta EE completo (ex.: WildFly).	A aplicação só precisa de um contêiner Servlet; o Nginx concentra TLS, estáticos e limitação de taxa com baixo consumo de recursos (NGINX, 2026; APACHE SOFTWARE FOUNDATION, 2026).	RNF01, RNF04, RNF07
DA04	Java com Spring Boot no servidor de aplicação.	Node.js; Python.	Ecossistema maduro para validação, tratamento de erros padronizado e acesso a dados; disponibilidade de implementação de Argon2id (SPRING, 2026).	RF02, RF07
DA05	PostgreSQL como repositório de usuários.	Diretório LDAP; banco NoSQL.	Restrições de unicidade, índices e controle fino de privilégios por coluna (POSTGRESQL GLOBAL DEVELOPMENT GROUP, 2026).	RF03, RNF06
DA06	Verificação de senha com Argon2id.	bcrypt; PBKDF2.	Função de custo elevado em memória, resistente a ataques com GPU; primeira recomendação da OWASP (BIRYUKOV *et al.*, 2021; OWASP, 2026b).	RF04, RNF02
DA07	Resposta de falha única e verificação contra *hash* fictício quando o usuário não existe.	Mensagens distintas para “usuário inexistente” e “senha incorreta”.	Evita a enumeração de usuários pelo conteúdo e pelo tempo da resposta (OWASP, 2026a).	RF07, RNF03
DA08	API sem estado e sem emissão de sessão ou *token*; a identidade é devolvida no corpo da resposta.	Sessão com *cookie*; *token* JWT.	Atende ao escopo, que exige apenas a identificação do usuário para que o cliente continue o fluxo.	RF06, RNF08
DA09	Erros no formato *Problem Details* (`application/problem+json`).	Formato de erro próprio.	Formato padronizado e legível por máquina (NOTTINGHAM; WILDE; DALAL, 2023).	RF02, RF07
DA10	Servidor organizado em camadas, com núcleo isolado por portas (interfaces) e adaptadores.	Controlador acessando o banco diretamente.	Dependências apontam para o núcleo, que pode ser testado sem servidor nem banco (MARTIN, 2019).	RNF10

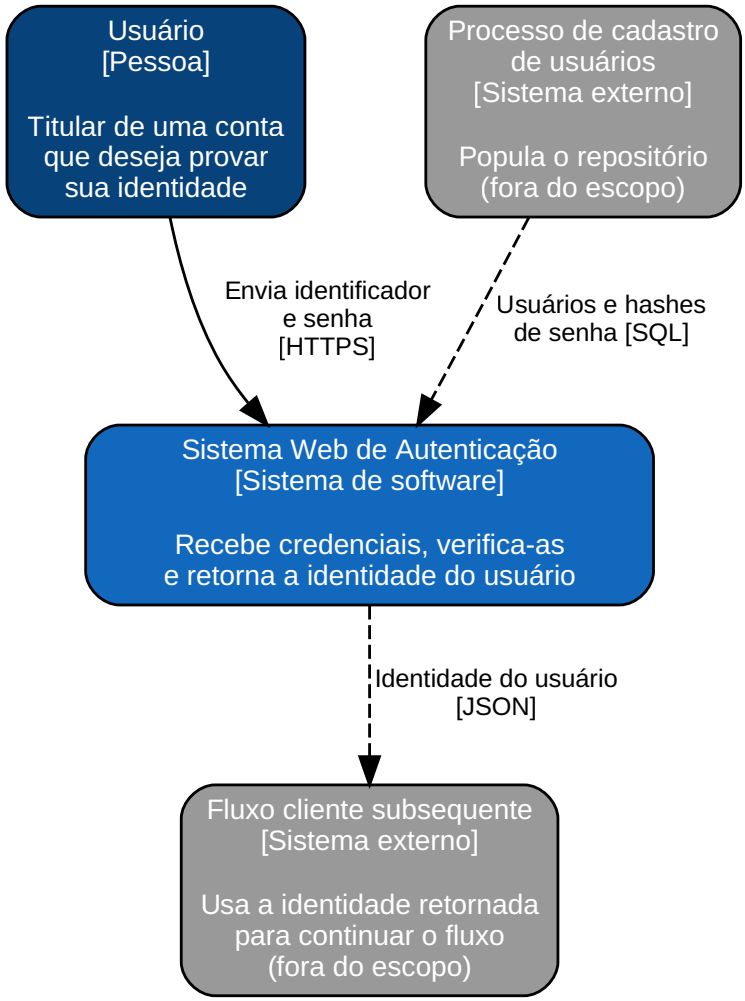
Fonte: elaborada pelos autores.

2.3 Visões de contexto e de contêineres

Item de informação 2.3 Autoria: Kezer dos Santos Souza Revisão: Gabriel Matos Mendes Apoio de IA: ver Nota

A visão de contexto (Figura 5) delimita o sistema e mostra com quem ele interage (BROWN, 2026). O **usuário** envia identificador e senha; o sistema devolve a identidade, que será usada por um **fluxo cliente subsequente**, fora do escopo. O **processo de cadastro**, também externo, é quem popula o repositório com os usuários e seus *hashes* de senha.

Figura 5 – Visão de contexto do Sistema Web de Autenticação (C4, nível 1)

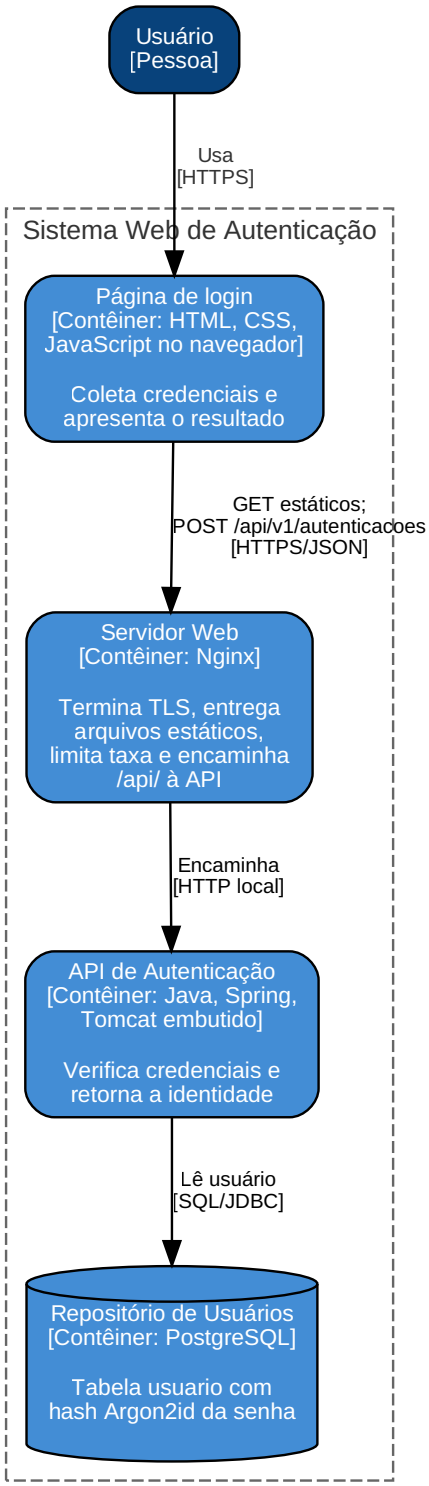


Fonte: elaborada pelos autores.

A visão de contêineres (Figura 6) decompõe o sistema em unidades executáveis ou de armazenamento independentes (BROWN, 2026). Cada contêiner corresponde a um elemento do ambiente operacional descrito na Seção 1:

- **Página de login**: HTML, CSS e JavaScript executados pelo navegador (item 1.4); coleta as credenciais e apresenta o resultado;
- **Servidor Web**: Nginx (item 1.5); termina o TLS, entrega a página, limita a taxa de tentativas e encaminha `/api/` à API;
- **API de Autenticação**: aplicação Spring Boot com Tomcat embutido, na JVM (item 1.5); verifica as credenciais e devolve a identidade;
- **Repositório de Usuários**: banco PostgreSQL com a tabela `usuario` (item 2.6).

Figura 6 – Visão de contêineres do Sistema Web de Autenticação (C4, nível 2)



Fonte: elaborada pelos autores.

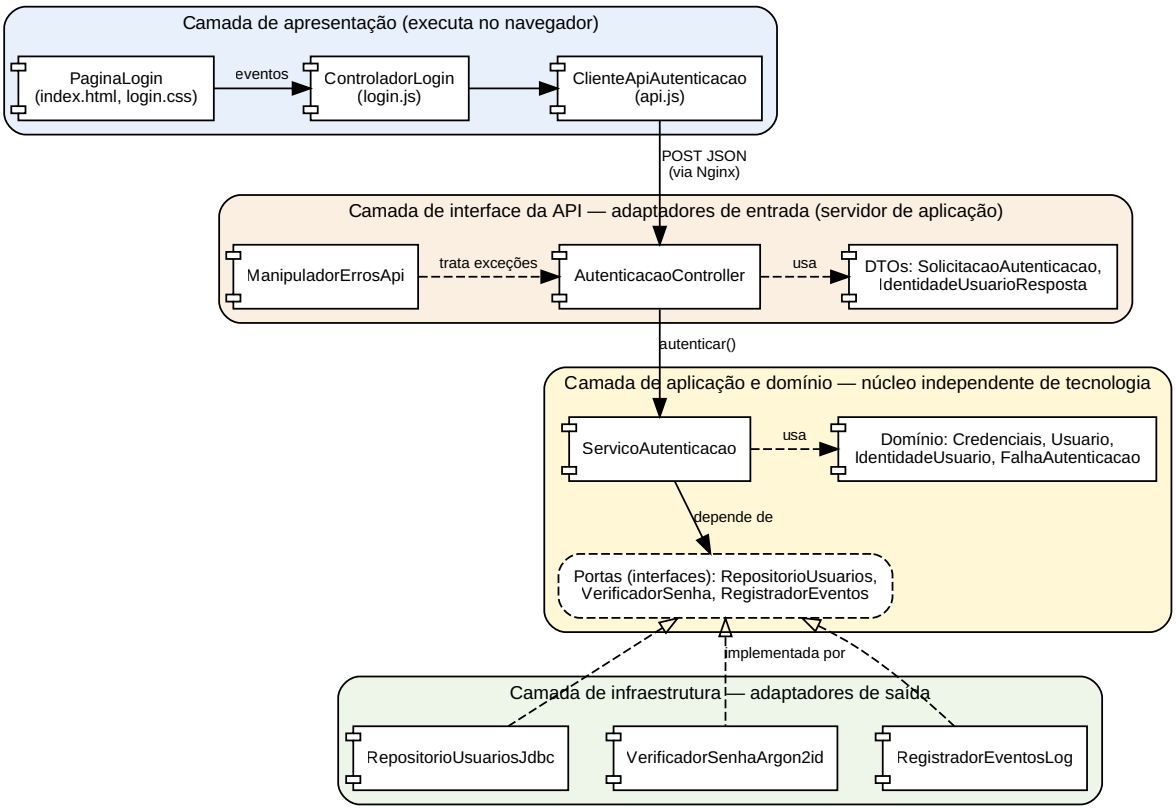
2.4 Visão de componentes

Item de informação 2.4 Autoria: Gabriel Matos Mendes Revisão: Kezer dos Santos Souza Apoio de IA: ver Nota

A Figura 7 apresenta os componentes da página de login e da API, organizados em camadas. No servidor, a regra de dependência é respeitada: as camadas externas dependem do núcleo, nunca o contrário. O núcleo declara **portas** — interfaces que expressam o que ele precisa do mundo exter-

no — e a camada de infraestrutura fornece **adaptadores** que as implementam, aplicando o princípio da inversão de dependência (MARTIN, 2019). O repositório segue o padrão *Repository*, que expõe ao domínio uma interface semelhante a uma coleção de objetos e esconde o SQL (FOWLER, 2002).

Figura 7 – Visão de componentes da página de login e da API de Autenticação



Fonte: elaborada pelos autores.

A Tabela 8 descreve a responsabilidade de cada componente e onde ele executa. O desenho interno das classes da API está no item 2.7.

Tabela 8 – Componentes, responsabilidades e local de execução

Componente	Camada	Responsabilidade	Executa em
PaginaLogin (`index.html`, `login.css`)	Apresentação	Formulário com campos `identificador` (`autocomplete="username"`) e `senha` (`autocomplete="current-password"`, colar permitido, opção de exibir a senha), botão Entrar e área de mensagens acessível (`aria-live`).	Navegador
ControladorLogin (`login.js`)	Apresentação	Trata o evento de envio, impede o envio padrão do formulário, desabilita o botão durante a requisição, exibe o resultado e, no sucesso, publica a identidade para o fluxo cliente subsequente.	Navegador
ClienteApiAutenticacao (`api.js`)	Apresentação	Envia `POST` com corpo JSON via `fetch()`, aplica tempo limite e traduz o código de *status* em resultado (autenticado, não autenticado ou erro).	Navegador
AutenticacaoController	Interface da API	Recebe a requisição HTTP, dispara a validação do corpo, chama o serviço e monta a resposta 200.	Servidor de aplicação
ManipuladorErrosApi	Interface da API	Converte exceções em respostas *Problem Details* (400, 401, 503, 500), sem expor detalhes internos.	Servidor de aplicação

Componente	Camada	Responsabilidade	Executa em
DTOs (SolicitacaoAutenticacao, IdentidadeUsuarioResposta)	Interface da API	Representam os corpos JSON de entrada e saída e suas regras de validação.	Servidor de aplicação
ServicoAutenticacao	Aplicação	Coordena o caso de uso: normaliza credenciais, busca o usuário, verifica a senha, decide, registra o evento e devolve a identidade ou lança a falha.	Servidor de aplicação
Domínio (Credenciais, Usuario, IdentidadeUsuario, FalhaAutenticacaoException)	Domínio	Conceitos do problema de autenticação, sem dependência de tecnologia.	Servidor de aplicação
Portas (RepositorioUsuarios, VerificadorSenha, RegistradorEventos)	Domínio	Interfaces das quais o serviço depende.	Servidor de aplicação
RepositorioUsuariosJdbc	Infraestrutura	Consulta a tabela `usuario` com SQL parametrizado.	Servidor de aplicação → SGBD
VerificadorSenhaArgon2id	Infraestrutura	Verifica a senha contra o *hash* Argon2id e fornece o *hash* fictício.	Servidor de aplicação
RegistradorEventosLog	Infraestrutura	Grava eventos de autenticação estruturados no *log* da aplicação.	Servidor de aplicação

Fonte: elaborada pelos autores.

2.5 Especificação da interface (API)

Item de informação 2.5 Autoria: Kezer dos Santos Souza Revisão: Gabriel Matos Mendes Apoio de IA: ver Nota

A API expõe um único recurso. O método `POST` foi escolhido porque as credenciais devem trafegar no corpo da mensagem — nunca na URL, que fica registrada em *logs* e no histórico do navegador — e porque a operação não é segura nem idempotente no sentido HTTP (FIELDING; NOTTINGHAM; RESCHKE, 2022a). Como nenhum recurso persistente é criado, o sucesso é indicado por `200 OK`, e não por `201 Created`. A Tabela 9 resume o contrato e a Tabela 10 define os campos da requisição, codificados em JSON (BRAY, 2017).

Tabela 9 – Contrato do recurso de autenticação

Elemento	Especificação
Recurso e método	`POST /api/v1/autenticacoes` (versão na URL para permitir evolução compatível)
Cabeçalhos de requisição	`Content-Type: application/json`; `Accept: application/json`
Tamanho máximo do corpo	4 KiB (verificado pelo Nginx)
Respostas	`application/json` no sucesso; `application/problem+json` nos erros (NOTTINGHAM; WILDE; DALAL, 2023); sempre com `Cache-Control: no-store`

Fonte: elaborada pelos autores.

Tabela 10 – Campos do corpo da requisição

Campo	Tipo	Obrigatório	Restrições
`identificador`	texto	sim	Nome de usuário ou e-mail; não vazio após remoção de espaços nas extremidades; no máximo 254 caracteres. Se contém “@”, é tratado como e-mail; caso contrário, como nome de usuário.

Campo	Tipo	Obrigatório	Restrições
`senha`	texto	sim	Não vazia; no máximo 256 unidades UTF-16 (limite de proteção contra corpos abusivos, suficiente para 128 caracteres Unicode quaisquer). Nenhuma regra de composição é aplicada na verificação.

Fonte: elaborada pelos autores.

Exemplo de requisição e de resposta de sucesso:

```
POST /api/v1/autenticacoes HTTP/1.1
Host: auth.example.com
Content-Type: application/json
Accept: application/json

{"identificador": "maria.silva", "senha": "correto cavalo bateria grampo"}

HTTP/1.1 200 OK
Content-Type: application/json
Cache-Control: no-store

{"id": "3f2b7c1e-8d4a-4c0e-9a51-6e2f0b9d7a10", "login": "maria.silva",
 "email": "maria.silva@example.com", "nomeExibicao": "Maria Silva",
 "autenticadoEm": "2026-10-27T18:30:00Z"}
```

Exemplo de resposta de falha de autenticação, idêntica para usuário inexistente e senha incorreta (RNF03). Como a especificação HTTP exige que toda resposta `401` traga o cabeçalho `WWW-Authenticate` (FIELDING; NOTTINGHAM; RESCHKE, 2022a), define-se o esquema próprio `CredenciaisJson`, que indica ao cliente que as credenciais devem ser enviadas no corpo JSON deste recurso:

```
HTTP/1.1 401 Unauthorized
Content-Type: application/problem+json
Cache-Control: no-store
WWW-Authenticate: CredenciaisJson realm="auth.example.com"

{"type": "https://auth.example.com/problemas/credenciais-invalidas",
 "title": "Falha de autenticação", "status": 401,
 "detail": "Identificador ou senha inválidos."}
```

A Tabela 11 lista todas as respostas possíveis, indicando qual software do ambiente operacional as produz.

Tabela 11 – Respostas da API e software que as produz

Status	Situação	Corpo e cabeçalhos	Produzida por
`200 OK`	Usuário existe e a senha confere (RF05, RF06).	Objeto de identidade.	API (Tomcat/ Spring)
`400 Bad Request`	JSON malformado ou campo ausente, vazio ou longo demais (RF02).	*Problem Details* com a lista de campos inválidos, sem ecoar os valores recebidos.	API
`401 Unauthorized`	Usuário inexistente ou senha incorreta (RF05, RF07).	*Problem Details* genérico; `WWW-Authenticate`.	API
`405 Method Not Allowed`	Método diferente de `POST`.	`Allow: POST`.	API
`413 Content Too Large`	Corpo maior que 4 KiB.	Página de erro do servidor web.	Nginx

Status	Situação	Corpo e cabeçalhos	Produzida por
`415 Unsupported Media Type`	Corpo que não é JSON.	*Problem Details*.	API
`429 Too Many Requests`	Limite de taxa por endereço IP excedido (RNF04).	Página de erro do servidor web.	Nginx
`500 Internal Server Error`	Falha inesperada na aplicação.	*Problem Details* genérico, sem rastreamento de pilha.	API
`502` / `504`	API inacessível ou sem resposta no tempo limite.	Página de erro do servidor web.	Nginx
`503 Service Unavailable`	Repositório de usuários indisponível.	*Problem Details*.	API

Fonte: elaborada pelos autores.

2.6 Modelo de dados

Item de informação 2.6 Autoria: Gabriel Matos Mendes Revisão: Kezer dos Santos Souza Apoio de IA: ver Nota

O repositório contém uma única tabela, `usuario` (Figura 8), com apenas os dados necessários para verificar credenciais e montar o objeto de identidade. O identificador primário é um UUID, que não revela a quantidade nem a ordem de cadastro dos usuários.

Figura 8 – Modelo físico da tabela de usuários

usuario		
PK	id	UUID NOT NULL
UK	login	VARCHAR(64) NOT NULL
UK	email	VARCHAR(254) NOT NULL
	nome_exibicao	VARCHAR(120) NOT NULL
	hash_senha	VARCHAR(255) NOT NULL
	criado_em	TIMESTAMPTZ NOT NULL
	atualizado_em	TIMESTAMPTZ NOT NULL

Fonte: elaborada pelos autores.

Restrições e índices garantem as premissas de que o projeto depende: unicidade de `login` e de `email` (cujas restrições criam os índices usados pela busca); armazenamento normalizado em minúsculas, que torna a busca insensível a maiúsculas; proibição de "@" no nome de usuário, que torna inequívoca a interpretação do identificador (Tabela 10); e formato Argon2id obrigatório para o *hash*. O papel de banco usado pela aplicação recebe apenas permissão de leitura nas colunas necessárias (RNF06) (POSTGRESQL GLOBAL DEVELOPMENT GROUP, 2026):

```
CREATE TABLE usuario (
  id             UUID          PRIMARY KEY,
  login          VARCHAR(64)   NOT NULL,
  email          VARCHAR(254)  NOT NULL,
  nome_exibicao  VARCHAR(120)  NOT NULL,
  hash_senha     VARCHAR(255)  NOT NULL,
  criado_em      TIMESTAMPTZ   NOT NULL DEFAULT now(),
  atualizado_em  TIMESTAMPTZ   NOT NULL DEFAULT now(),
  CONSTRAINT uk_usuario_login   UNIQUE (login),
  CONSTRAINT uk_usuario_email   UNIQUE (email),
  CONSTRAINT ck_login_normal    CHECK (login = lower(login) AND position('@' IN login)
= 0),
  CONSTRAINT ck_email_normal    CHECK (email = lower(email)),
  CONSTRAINT ck_hash_argon2id   CHECK (hash_senha LIKE '$argon2id$%')
);
CREATE ROLE app_autenticacao LOGIN;   -- senha definida fora do repositório
GRANT SELECT (id, login, email, nome_exibicao, hash_senha) ON usuario TO app_autentica-
cao;
```

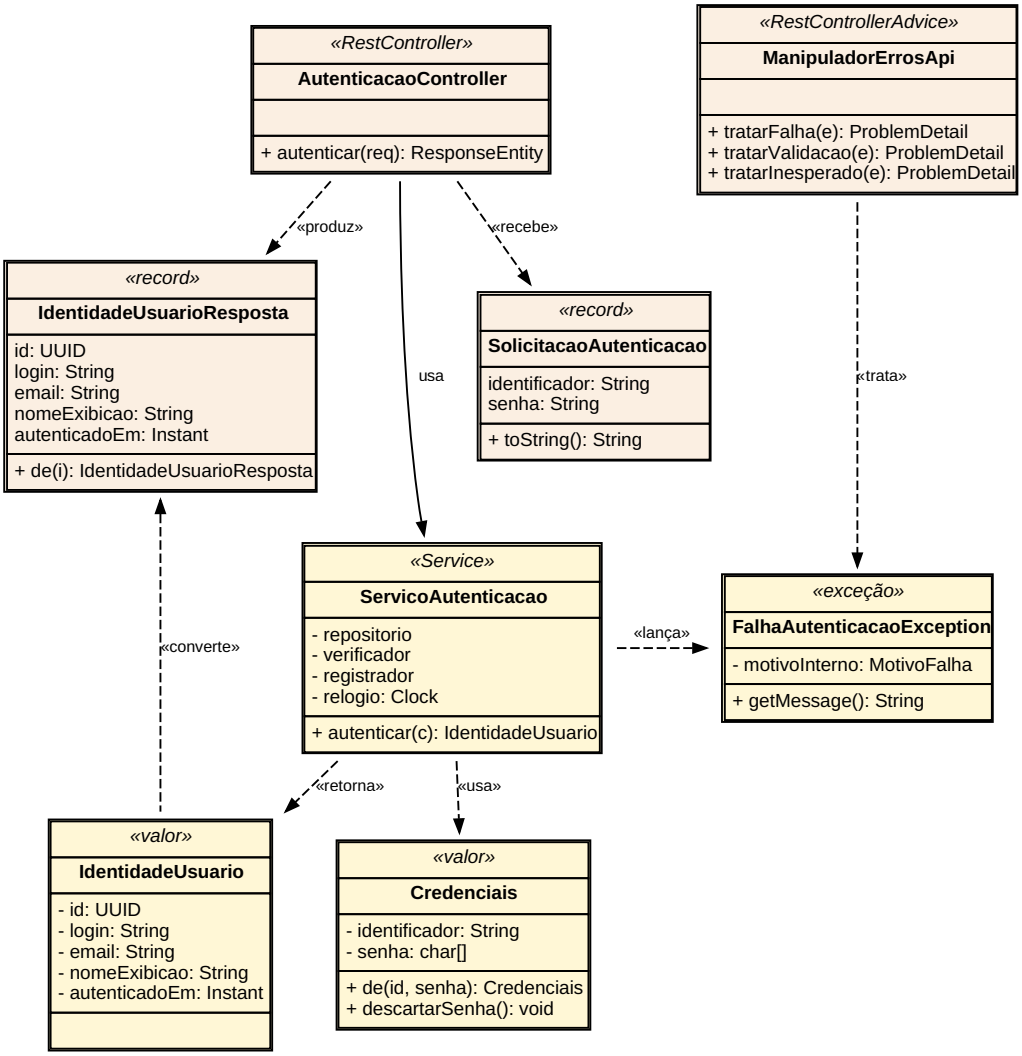
A coluna `hash_senha` armazena a cadeia no formato PHC, que reúne algoritmo, versão, parâmetros, sal e *hash*, por exemplo `$argon2id$v=19$m=19456,t=2,p=1$<sal>$<hash>`. Como os parâmetros ficam junto de cada *hash*, o custo pode ser elevado no futuro sem invalidar os registros existentes (BIRYUKOV *et al.*, 2021; OWASP, 2026b).

2.7 Desenho detalhado dos componentes

Item de informação 2.7 Autoria: Kezer dos Santos Souza Revisão: Gabriel Matos Mendes Apoio de IA: ver Nota

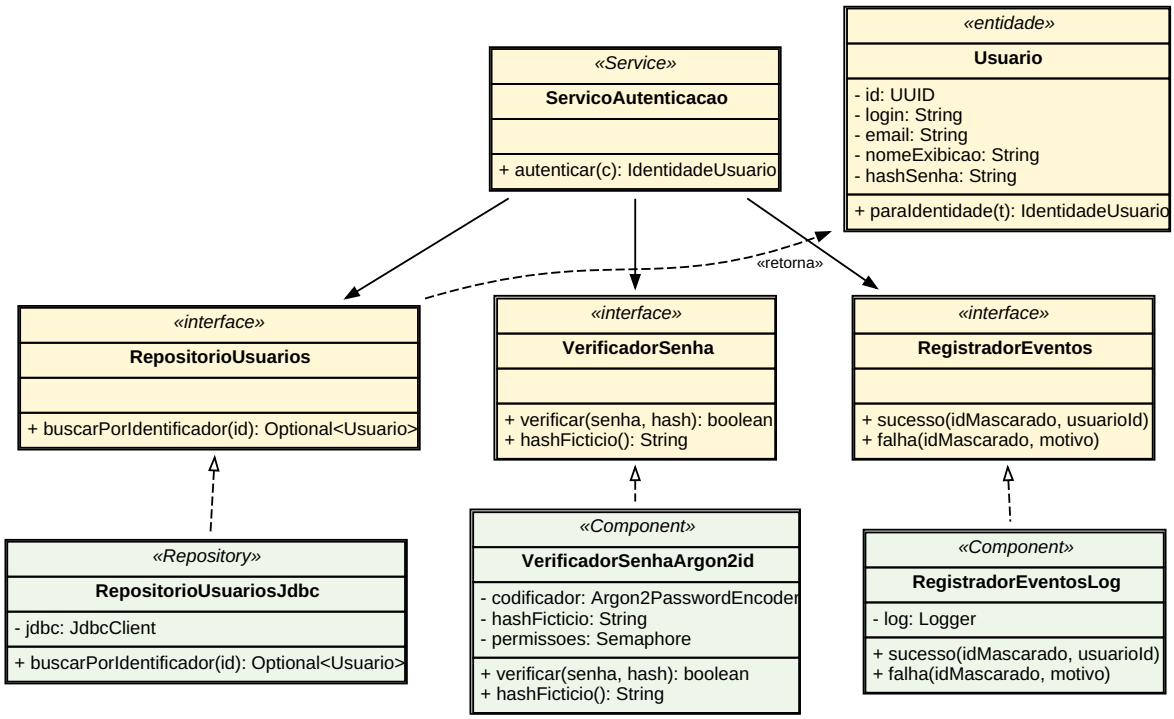
O desenho detalhado da API é apresentado em dois diagramas de classes UML (OMG, 2017): a Figura 9 mostra a interface da API, o serviço e os objetos de domínio que trafegam entre eles; a Figura 10 mostra as portas do núcleo e os adaptadores de infraestrutura que as implementam. Estereótipos indicam o papel de cada classe no Spring (por exemplo, «RestController», «Service»).

Figura 9 – Diagrama de classes – interface da API, serviço e domínio



Fonte: elaborada pelos autores.

Figura 10 – Diagrama de classes – portas do núcleo e adaptadores de infraestrutura



Fonte: elaborada pelos autores.

A Tabela 12 especifica as operações de cada classe, com assinaturas completas e as regras que cada uma deve cumprir.

Tabela 12 – Especificação das classes da API

Classe	Operações (assinatura)	Regras de desenho
AutenticacaoController	`ResponseEntity<IdentidadeUsuarioResposta> autenticar(@Valid @RequestBody SolicitacaoAutenticacao req)`	Mapeado para `POST /api/v1/autenticacoes`, consome e produz JSON. Cria `Credenciais.de(req.identificador(), req.senha())`, chama o serviço e responde `200` com `Cache-Control: no-store`. Não contém regra de negócio.
ManipuladorErrosApi	`ProblemDetail tratarFalha(FalhaAutenticacaoException e); tratarValidacao(MethodArgumentNotValidException e); tratarIndisponibilidade(DataAccessException e); tratarInesperado(Exception e)`	Produz respectivamente 401 (com `WWW-Authenticate`), 400, 503 e 500, nos formatos do item 2.5. Nunca inclui a senha, o motivo interno da falha ou o rastreamento de pilha.
SolicitacaoAutenticacao	*record* `(String identificador, String senha); String toString()`	Anotações `@NotBlank @Size(max = 254)` e `@NotEmpty @Size(max = 256)`. `toString()` mascara a senha, impedindo seu registro acidental em *logs* (RNF05).
IdentidadeUsuarioResposta	*record* `(UUID id, String login, String email, String nomeExibicao, Instant autenticadoEm); static de(IdentidadeUsuario i)`	Contém somente dados de identificação; nunca o *hash*.

Classe	Operações (assinatura)	Regras de desenho
ServicoAutenticacao	`IdentidadeUsuario autenticar(Credenciais c) throws FalhaAutenticacaoException`	Implementa o algoritmo do item 2.8. Depende apenas das portas e de um `Clock` (para o instante da autenticação, o que torna o serviço testável). Não guarda estado entre chamadas.
Credenciais	`static Credenciais de(String identificador, String senha)`; `String identificador()`; `char[] senha()`; `void descartarSenha()`	Remove espaços das extremidades do identificador e o converte para minúsculas (`Locale.ROOT`); normaliza a senha em Unicode NFKC, como recomenda a NIST (TEMOSHOK *et al.*, 2025), e a guarda em `char[]`, que `descartarSenha()` sobrescreve com zeros.
Usuario	`IdentidadeUsuario paraIdentidade(Instant instante)`	Entidade lida do repositório, com o *hash*; o *hash* não sai do núcleo.
IdentidadeUsuario	valor imutável (`id, login, email, nomeExibicao, autenticadoEm`)	Resultado do caso de uso em caso de sucesso (RF06).
FalhaAutenticacaoException	`MotivoFalha motivoInterno()`; `String getMessage()`	`MotivoFalha` ∈ {`USUARIO_INEXISTENTE`, `SENHA_INCORRETA`} serve apenas ao registro interno; a mensagem é sempre a mesma (RF07).
RepositorioUsuarios / RepositorioUsuariosJdbc	`Optional<Usuario> buscarPorIdentificador(String identificador)`	Escolhe a consulta pela presença de "@": `SELECT id, login, email, nome_exibicao, hash_senha FROM usuario WHERE email = ?` ou `… WHERE login = ?`. Sempre parametrizada, o que impede injeção de SQL.
VerificadorSenha / VerificadorSenhaArgon2id	`boolean verificar(char[] senha, String hash)`; `String hashFicticio()`	Usa `Argon2PasswordEncoder` (módulo `spring-security-crypto`) com sal de 16 bytes, *hash* de 32 bytes, memória de 19 MiB, 2 iterações e paralelismo 1; os parâmetros de cada verificação são lidos da própria cadeia armazenada. O *hash* fictício é gerado na inicialização a partir de um valor aleatório, com os mesmos parâmetros. Um semáforo limita a 8 as verificações simultâneas, contendo o consumo de memória (8 × 19 MiB) (RNF07).
RegistradorEventos / RegistradorEventosLog	`void sucesso(String idMascarado, UUID usuarioId)`; `void falha(String idMascarado, MotivoFalha motivo)`	Grava eventos JSON com data, resultado e identificador mascarado (ex.: `ma***@ex***`). O endereço IP é obtido pela correlação com o *log* do Nginx, que já o registra.

Fonte: elaborada pelos autores.

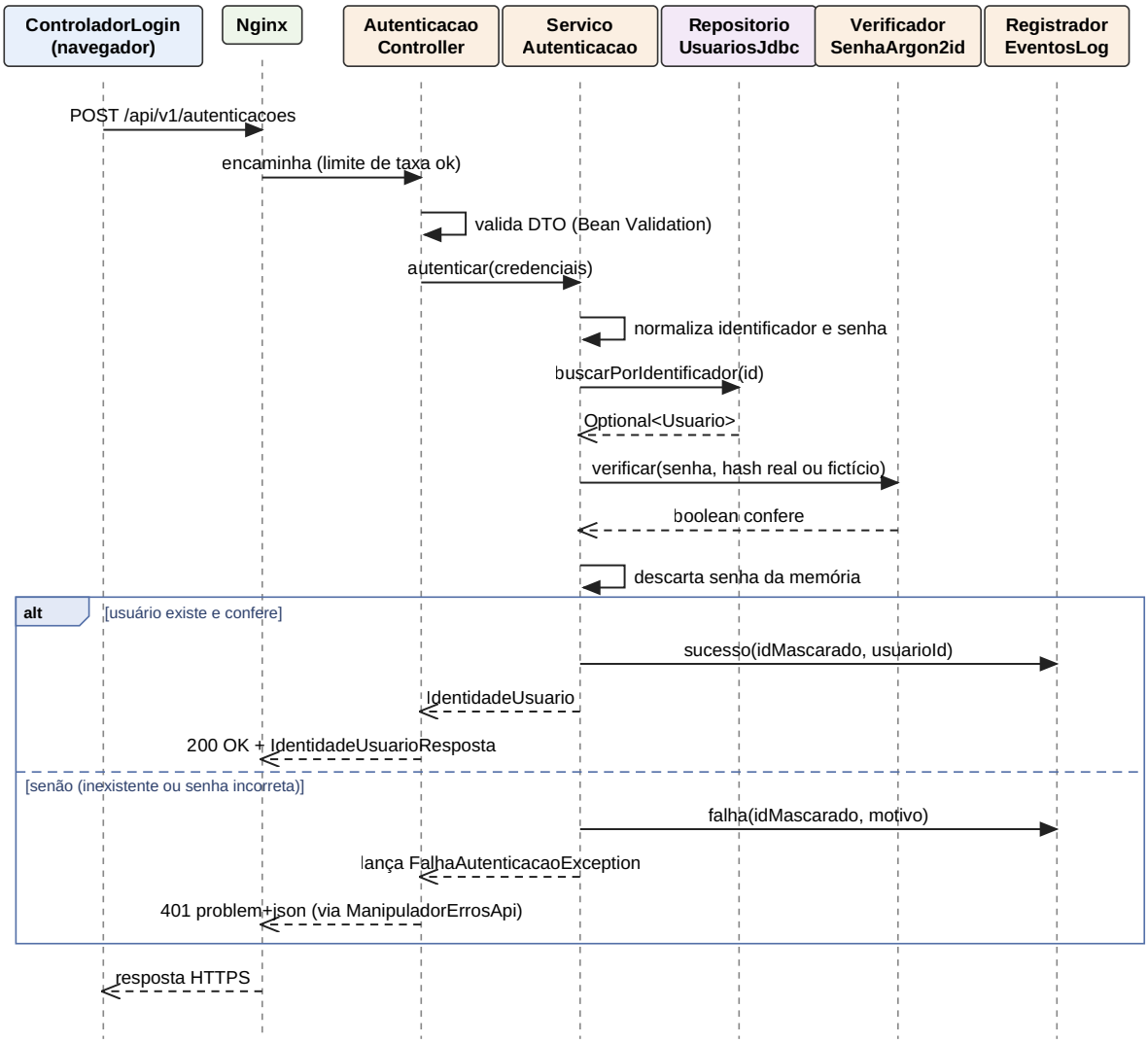
Componentes do navegador. `ClienteApiAutenticacao.autenticar(identificador, senha)` devolve uma *Promise* que se resolve em um de três resultados: `{tipo: "AUTENTICADO", identidade}` para `200`, `{tipo: "NAO_AUTENTICADO"}` para `401` e `{tipo: "ERRO", mensagem}` para as demais respostas ou falhas de rede, com tempo limite de 10 s implementado com `AbortController` (WHATWG, 2026b). `ControladorLogin` apresenta a mensagem correspondente e, no sucesso, limpa o campo de senha e emite o evento `usuario-autenticado` com a identidade, por meio do qual o fluxo cliente subsequente (Figura 5) continua sem que a página de login precise conhecê-lo.

2.8 Comportamento dinâmico da autenticação

Item de informação 2.8 Autoria: Gabriel Matos Mendes Revisão: Kezer dos Santos Souza Apoio de IA: ver Nota

A Figura 11 mostra a colaboração entre os componentes durante uma autenticação, com o fragmento `alt` separando sucesso e falha; a Figura 12 mostra o fluxo de decisão do serviço (OMG, 2017). Enquanto a Figura 2 (item 1.3) descreve a requisição do ponto de vista dos softwares de apoio, estes diagramas descrevem o que acontece *dentro* da aplicação.

Figura 11 – Sequência de autenticação entre os componentes da solução

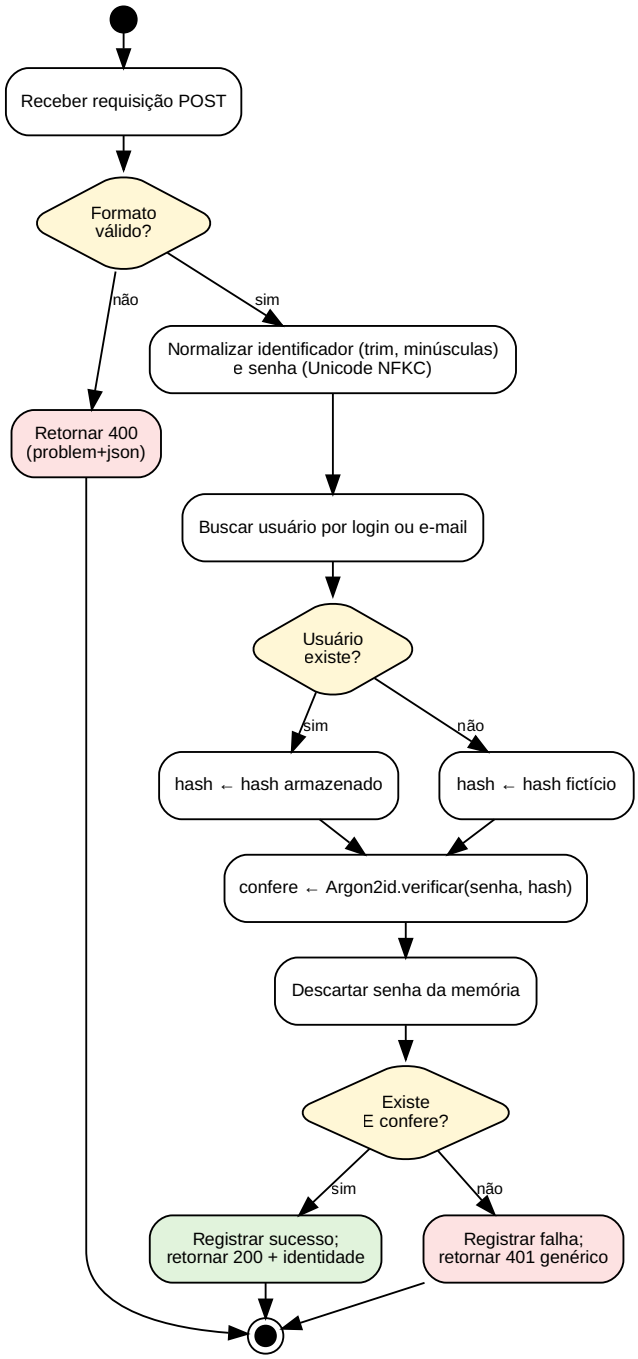


Fonte: elaborada pelos autores.

O algoritmo do método `ServicoAutenticacao.autenticar` é:

```
autenticar(c: Credenciais): IdentidadeUsuario
  id      ← c.identificador()                       // já normalizado (2.7)
  usuario ← repositorio.buscarPorIdentificador(id)  // Optional
  hash    ← usuario existe ? usuario.hashSenha : verificador.hashFicticio()
  confere ← verificador.verificar(c.senha(), hash)  // sempre executado
  c.descartarSenha()
  se usuario existe E confere:
      registrador.sucesso(mascarar(id), usuario.id)
      retornar usuario.paraIdentidade(relogio.instant())
  motivo ← usuario existe ? SENHA_INCORRETA : USUARIO_INEXISTENTE
  registrador.falha(mascarar(id), motivo)
  lançar FalhaAutenticacaoException(motivo)          // resposta 401 genérica
```

Figura 12 – Diagrama de atividades da decisão de autenticação



Fonte: elaborada pelos autores.

Equalização de tempo. Se o serviço retornasse imediatamente quando o usuário não existe, a resposta seria dezenas de milissegundos mais rápida do que quando a senha é verificada, e um atacante poderia descobrir quais identificadores existem medindo o tempo de resposta. Por isso a verificação Argon2id é sempre executada — contra o *hash* real ou contra o fictício, gerado com os mesmos parâmetros —, de modo que os dois caminhos tenham custo semelhante (RNF03) (OWASP, 2026a). A comparação final do *hash* é feita pela biblioteca em tempo constante.

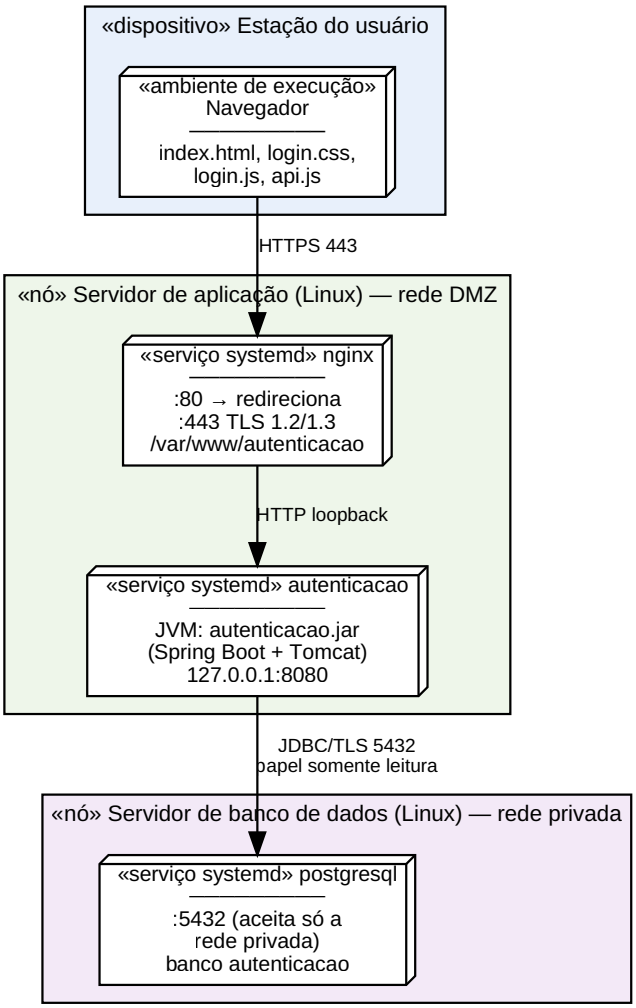
Tratamento de exceções. A validação do corpo falha antes de o serviço ser chamado e resulta em `400`; a falha de autenticação resulta em `401`; uma falha de acesso ao banco, como conexão recusada ou tempo esgotado, resulta em `503`; qualquer outra exceção, em `500`. Em todos os casos, o mapeamento é feito pelo `ManipuladorErrosApi`, conforme a Tabela 11.

2.9 Implantação e configuração do ambiente operacional

Item de informação 2.9 Autoria: Kezer dos Santos Souza Revisão: Gabriel Matos Mendes Apoio de IA: ver Nota

A Figura 13 mostra a implantação dos contêineres nos nós físicos ou virtuais. O servidor de aplicação escuta apenas a interface de *loopback* e só é alcançado pelo Nginx; o SGBD fica em rede privada e aceita conexões cifradas apenas do servidor de aplicação (RNF06).

Figura 13 – Diagrama de implantação



Fonte: elaborada pelos autores.

Servidor web. O trecho de configuração do Nginx abaixo materializa as decisões DA02 e DA03 e os requisitos RNF01 e RNF04: redireciona HTTP para HTTPS, aceita apenas TLS 1.2 e 1.3, envia os cabeçalhos HSTS e de política de conteúdo, entrega os arquivos da página, limita a 10 tentativas por minuto por endereço IP (com rajada de 5) e encaminha a API ao servidor de aplicação com tempos limite explícitos (NGINX, 2026; HODGES; JACKSON; BARTH, 2012):

```
limit_req_zone $binary_remote_addr zone=autenticacao:10m rate=10r/m;

server { listen 80; server_name auth.example.com; return 301 https://$host$re-
quest_uri; }

server {
  listen 443 ssl;  http2 on;  server_name auth.example.com;
  ssl_certificate     /etc/ssl/auth/fullchain.pem;
  ssl_certificate_key /etc/ssl/auth/privkey.pem;
  ssl_protocols       TLSv1.2 TLSv1.3;
  add_header Strict-Transport-Security "max-age=31536000; includeSubDomains" always;
  add_header Content-Security-Policy "default-src 'self'; frame-ancestors 'none'"
always;
  add_header X-Content-Type-Options nosniff always;
  root /var/www/autenticacao;

  location / { try_files $uri $uri/ =404; }

  location = /api/v1/autenticacoes {
    limit_req zone=autenticacao burst=5 nodelay;  limit_req_status 429;
    client_max_body_size 4k;
    proxy_pass http://127.0.0.1:8080;
    proxy_set_header Host              $host;
    proxy_set_header X-Forwarded-For   $proxy_add_x_forwarded_for;
    proxy_set_header X-Forwarded-Proto $scheme;
    proxy_connect_timeout 2s;  proxy_read_timeout 10s;
  }
}
```

Servidor de aplicação. A configuração da aplicação Spring Boot define a interface de escuta, o tratamento dos cabeçalhos repassados pelo Nginx, o tamanho dos *pools* e a conexão cifrada com o banco. A senha do banco é injetada por variável de ambiente e nunca é versionada (SPRING, 2026):

```
server:
  address: 127.0.0.1
  port: 8080
  forward-headers-strategy: native   # usa X-Forwarded-* do Nginx
  tomcat.threads.max: 50
spring:
  datasource:
    url: jdbc:postgresql://bd.interno:5432/autenticacao?sslmode=verify-full
    username: app_autenticacao
    password: ${DB_SENHA}
    hikari.maximum-pool-size: 10
```

Inicialização. Os três serviços são unidades do `systemd`, iniciadas nesta ordem: `postgresql`, `autenticacao` e `nginx`. A unidade da aplicação executa `/usr/bin/java -Xms512m -Xmx512m -jar /opt/autenticacao/autenticacao.jar` com um usuário de sistema sem privilégios, lê `DB_SENHA` de um arquivo de ambiente legível apenas por esse usuário e é reiniciada automaticamente em caso de falha. A memória máxima da JVM comporta as 8 verificações Argon2id simultâneas definidas no item 2.7. Antes de recarregar o Nginx, a configuração é validada com `nginx -t`.

2.10 Rastreabilidade entre requisitos, projeto e ambiente

Item de informação 2.10 Autoria: Gabriel Matos Mendes Revisão: Kezer dos Santos Souza Apoio de IA: ver Nota

A Tabela 13 relaciona cada requisito do item 2.1 aos elementos de projeto que o atendem, aos itens onde esses elementos são especificados e aos elementos do ambiente operacional da Seção 1 envolvidos. Ela permite verificar a completude do projeto — todo requisito é atendido por algum elemento — e avaliar o impacto de mudanças (PRESSMAN; MAXIM, 2021).

Tabela 13 – Matriz de rastreabilidade

Req.	Elementos de projeto	Itens	Ambiente (Seção 1)
RF01	PaginaLogin, ControladorLogin, ClienteApiAutenticacao, AutenticacaoController, SolicitacaoAutenticacao	item 2.4, item 2.5, item 2.7	Navegador (item 1.4); Nginx e Tomcat (item 1.5)
RF02	Validação de SolicitacaoAutenticacao, ManipuladorErrosApi	item 2.5, item 2.7	Spring (item 1.5)
RF03	RepositorioUsuarios(Jdbc), tabela `usuario`	item 2.6, item 2.7, item 2.8	JDBC e PostgreSQL (item 1.5)
RF04	VerificadorSenha(Argon2id)	item 2.7, item 2.8	JVM (item 1.2)
RF05	ServicoAutenticacao	item 2.8	Tomcat (item 1.5)
RF06	IdentidadeUsuario, IdentidadeUsuarioResposta, AutenticacaoController	item 2.5, item 2.7	Spring (item 1.5); navegador (item 1.4)
RF07	FalhaAutenticacaoException, ManipuladorErrosApi	item 2.5, item 2.8	Spring (item 1.5)
RNF01	Configuração TLS, redirecionamento e HSTS do Nginx; DA02	item 2.2, item 2.9	TLS (item 1.3); Nginx (item 1.5)
RNF02	VerificadorSenhaArgon2id; restrição `ck_hash_argon2id`	item 2.6, item 2.7	PostgreSQL (item 1.5)
RNF03	*Hash* fictício em ServicoAutenticacao; resposta 401 única	item 2.5, item 2.8	—
RNF04	`limit_req` do Nginx	item 2.9	Nginx (item 1.5)
RNF05	RegistradorEventosLog; `toString()` mascarado	item 2.7	*Logs* (item 1.5)
RNF06	Papel somente leitura; escuta em *loopback*; banco em rede privada	item 2.6, item 2.9	Tomcat e PostgreSQL (item 1.2)
RNF07	Semáforo do verificador; *pools* de *threads* e conexões	item 2.7, item 2.9	Tomcat e JDBC (item 1.5)
RNF08	API sem estado (DA08)	item 2.2, item 2.3	Nginx como *proxy* (item 1.5)
RNF09	Normalização NFKC em Credenciais; atributos `autocomplete` da PaginaLogin	item 2.4, item 2.7	Navegador (item 1.4)
RNF10	Camadas com portas e adaptadores (DA10)	item 2.2, item 2.4	—

Fonte: elaborada pelos autores.

REFERÊNCIAS

ANTHROPIC. **Claude Opus 5.5**: modelo de linguagem de grande escala. San Francisco: Anthropic, 2026. Assistente de IA utilizado na elaboração deste documento. Disponível em: https://claude.ai. Acesso em: 27 out. 2026.

APACHE SOFTWARE FOUNDATION. **Apache Tomcat documentation**. Wilmington: ASF, 2026. Disponível em: https://tomcat.apache.org/. Acesso em: 27 out. 2026.

BASS, L.; CLEMENTS, P.; KAZMAN, R. **Software architecture in practice**. 4. ed. Boston: Addison-Wesley, 2021.

BIRYUKOV, A.; DINU, D.; KHOVRATOVICH, D.; JOSEFSSON, S. **RFC 9106**: Argon2 memory-hard function for password hashing and proof-of-work applications. [*S. l.*]: IRTF, set. 2021. DOI: 10.17487/RFC9106.

BRAY, T. (ed.). **RFC 8259**: The JavaScript Object Notation (JSON) data interchange format. [*S. l.*]: IETF, dez. 2017. DOI: 10.17487/RFC8259.

BROWN, S. **The C4 model for visualising software architecture**. [*S. l.*], 2026. Disponível em: https://c4model.com. Acesso em: 27 out. 2026.

ECLIPSE FOUNDATION. **Jakarta Servlet Specification**: version 6.0. Ottawa: Eclipse Foundation, 2022. Disponível em: https://jakarta.ee/specifications/servlet/6.0/. Acesso em: 27 out. 2026.

FIELDING, R. T. **Architectural styles and the design of network-based software architectures**. 2000. Tese (Doutorado em Ciência da Informação e da Computação) – University of California, Irvine, 2000.

FIELDING, R.; NOTTINGHAM, M.; RESCHKE, J. (ed.). **RFC 9110**: HTTP semantics. [*S. l.*]: IETF, jun. 2022a. DOI: 10.17487/RFC9110.

FIELDING, R.; NOTTINGHAM, M.; RESCHKE, J. (ed.). **RFC 9112**: HTTP/1.1. [*S. l.*]: IETF, jun. 2022b. DOI: 10.17487/RFC9112.

FOWLER, M. **Patterns of enterprise application architecture**. Boston: Addison-Wesley, 2002.

GRIGORIK, I. **High performance browser networking**. Sebastopol: O'Reilly, 2013.

HODGES, J.; JACKSON, C.; BARTH, A. **RFC 6797**: HTTP Strict Transport Security (HSTS). [*S. l.*]: IETF, nov. 2012. DOI: 10.17487/RFC6797.

INTERNATIONAL ORGANIZATION FOR STANDARDIZATION; INTERNATIONAL ELECTROTECHNICAL COMMISSION. **ISO/IEC 25010:2023**: systems and software engineering – systems and software quality requirements and evaluation (SQuaRE) – product quality model. Genebra: ISO, 2023.

INTERNATIONAL ORGANIZATION FOR STANDARDIZATION; INTERNATIONAL ELECTROTECHNICAL COMMISSION; INSTITUTE OF ELECTRICAL AND ELECTRONICS ENGINEERS. **ISO/IEC/IEEE 12207:2026**: systems and software engineering – software life cycle processes. Genebra: ISO, 2026.

KUROSE, J. F.; ROSS, K. W. **Redes de computadores e a Internet**: uma abordagem *top-down*. 8. ed. São Paulo: Pearson, 2021.

MARTIN, R. C. **Arquitetura limpa**: o guia do artesão para estrutura e design de software. Rio de Janeiro: Alta Books, 2019.

MOCKAPETRIS, P. **RFC 1034**: domain names – concepts and facilities. [*S. l.*]: IETF, nov. 1987. DOI: 10.17487/RFC1034.

NGINX. **nginx documentation**. [*S. l.*]: F5, 2026. Disponível em: https://nginx.org/en/docs/. Acesso em: 27 out. 2026.

NOTTINGHAM, M.; WILDE, E.; DALAL, S. **RFC 9457**: problem details for HTTP APIs. [*S. l.*]: IETF, jul. 2023. DOI: 10.17487/RFC9457.

OBJECT MANAGEMENT GROUP. **OMG Unified Modeling Language (OMG UML)**: version 2.5.1. Needham: OMG, 2017. Disponível em: https://www.omg.org/spec/UML/2.5.1. Acesso em: 27 out. 2026.

OWASP FOUNDATION. **Authentication cheat sheet**. [*S. l.*]: OWASP, 2026a. Disponível em: https://cheatsheetseries.owasp.org/cheatsheets/Authentication_Cheat_Sheet.html. Acesso em: 27 out. 2026.

OWASP FOUNDATION. **Password storage cheat sheet**. [*S. l.*]: OWASP, 2026b. Disponível em: https://cheatsheetseries.owasp.org/cheatsheets/Password_Storage_Cheat_Sheet.html. Acesso em: 27 out. 2026.

POSTGRESQL GLOBAL DEVELOPMENT GROUP. **PostgreSQL documentation**. [*S. l.*], 2026. Disponível em: https://www.postgresql.org/docs/current/. Acesso em: 27 out. 2026.

PRESSMAN, R. S.; MAXIM, B. R. **Engenharia de software**: uma abordagem profissional. 9. ed. Porto Alegre: AMGH, 2021.

RESCORLA, E. **RFC 8446**: the Transport Layer Security (TLS) protocol version 1.3. [*S. l.*]: IETF, ago. 2018. DOI: 10.17487/RFC8446.

SOMMERVILLE, I. **Engenharia de software**. 10. ed. São Paulo: Pearson, 2019.

SPRING. **Spring Framework reference documentation**. [*S. l.*]: Broadcom, 2026. Disponível em: https://docs.spring.io/spring-framework/reference/. Acesso em: 27 out. 2026.

TEMOSHOK, D. *et al*. **Digital identity guidelines**: authentication and authenticator management. Gaithersburg: National Institute of Standards and Technology, jul. 2025. (NIST Special Publication 800-63B-4). DOI: 10.6028/NIST.SP.800-63B-4.

THOMSON, M.; BENFIELD, C. (ed.). **RFC 9113**: HTTP/2. [*S. l.*]: IETF, jun. 2022. DOI: 10.17487/RFC9113.

WHATWG. **Fetch living standard**. [*S. l.*]: WHATWG, 2026b. Disponível em: https://fetch.spec.whatwg.org/. Acesso em: 27 out. 2026.

WHATWG. **HTML living standard**. [*S. l.*]: WHATWG, 2026a. Disponível em: https://html.spec.whatwg.org/. Acesso em: 27 out. 2026.

APÊNDICE A – REGISTRO DE COMUNICAÇÕES E REUNIÕES DA EQUIPE

Item de informação
Autoria: Gabriel Matos Mendes Revisão: Kezer dos Santos Souza Apoio de IA: ver Nota

Este apêndice reúne os registros de comunicação e de atividades da equipe, exigidos pelo critério de interação com a equipe da avaliação individual. A comunicação ocorre em servidor do Discord com a participação da docente; as reuniões síncronas seguem o plano do Apêndice C. Para cada registro, a coluna “Evidência” aponta para a mensagem, ata ou captura de tela correspondente, mantida no repositório (Apêndice B).

A preencher pela equipe: os registros abaixo devem reproduzir comunicações e reuniões que de fato ocorreram (datas, participantes, decisões e evidências). Eles não foram gerados com apoio de IA, para não criar registros que não correspondam a fatos.

Tabela 14 – Registro de comunicações e reuniões

Data	Meio	Participantes	Assunto, decisões e encaminhamentos	Evidência
[dd/mm/2026]	[Discord / reunião presencial / aula prática]	[nomes]	[pauta	decisões e encaminhamentos]
[dd/mm/2026]	[ ]	[ ]	[ ]	[ ]
[dd/mm/2026]	[ ]	[ ]	[ ]	[ ]

Fonte: elaborada pelos autores.

APÊNDICE B – GERÊNCIA DE CONFIGURAÇÃO NO GITHUB

Item de informação
Autoria: Kezer dos Santos Souza Revisão: Gabriel Matos Mendes Apoio de IA: ver Nota

Todos os artefatos deste documento estão sob gerência de configuração no repositório `https://github.com/gabrielmmendes/aps-autenticacao-web`, no qual a docente (adriana.silveira.souza@gmail.com) foi incluída como colaboradora. O documento é mantido como código-fonte: o texto, as figuras e o PDF são gerados a partir de arquivos versionados, de modo que cada alteração é rastreável até seu autor. A Tabela 15 descreve a estrutura do repositório.

Tabela 15 – Estrutura do repositório

Caminho	Conteúdo
`conteudo.html`	Texto de todos os itens de informação, com macros de citação e remissão.
`diagramas/`	Fontes das figuras (Graphviz e gerador de diagramas de sequência).
`build.py`, `estilo.css`	Montagem do PDF: numeração de figuras e tabelas, referências, quadro de autoria e formatação padronizada.
`registros_comunicacao.csv`	Registros de comunicação apresentados no Apêndice A.
`build/APS_Autenticacao_Web.pdf`	Documento entregue, gerado a cada versão.

Fonte: elaborada pelos autores.

Fluxo de trabalho. Cada item de informação corresponde a uma *issue* no quadro Kanban do GitHub Projects, com as colunas *A fazer*, *Em autoria*, *Em revisão* e *Concluído*. O autor trabalha em um ramo próprio (por exemplo, `item/2.5-api`) e abre um *pull request* para o ramo `main`, indicando o revisor definido na Tabela 1. O revisor registra seus comentários no *pull request* e só ele pode aprová-lo; o ramo `main` é protegido contra integração sem aprovação. Assim, o histórico do repositório comprova a autoria e a revisão de cada item, e o quadro Kanban permite à equipe e à docente acompanhar continuamente o andamento dos compromissos individuais (Apêndice C).

APÊNDICE C – PLANEJAMENTO DO TRABALHO E GESTÃO DE RISCOS DA EQUIPE

Item de informação
Autoria: Gabriel Matos Mendes Revisão: Kezer dos Santos Souza Apoio de IA: ver Nota

O trabalho foi decomposto em unidades lógicas — os itens de informação —, e não em fatias de texto, segundo o princípio de "dividir e conquistar" do Apêndice C do enunciado da APS. A integração é garantida por dois mecanismos: a revisão cruzada, em que o autor de um item da Seção 1 revisa o item da Seção 2 que dele depende (Tabela 1), e a matriz de rastreabilidade (item 2.10).

Atividades síncronas e assíncronas. As atividades síncronas, que exigem a presença de todos, são a reunião de planejamento, a revisão de integração do documento e o ensaio da apresentação; a equipe estabelece por consenso no mínimo uma e no máximo duas atividades síncronas por semana, podendo usar as aulas práticas da disciplina. As atividades assíncronas — autoria e revisão de itens — são realizadas por cada integrante no horário de sua conveniência, com prazos registrados nas *issues* do quadro Kanban (Apêndice B).

Gestão de riscos do trabalho em equipe. A Tabela 16 registra os riscos identificados, com destaque para o risco de não cumprimento de compromisso individual. A probabilidade de cada risco é reavaliada a cada atividade síncrona, e não apenas na data de entrega; desvios são registrados no Apêndice A e, quando envolvem conduta ética ou profissional, comunicados à docente.

Tabela 16 – Registro de riscos do trabalho em equipe

Id.	Risco	Prob. / impacto	Monitoramento (indicador)	Resposta planejada
R1	Integrante não cumpre o compromisso de autoria ou revisão no prazo.	Média / alto	*Issue* sem movimentação por mais de 2 dias ou fora da coluna esperada no quadro Kanban.	Contato registrado no Discord; replanejamento do prazo; se persistir, redistribuição do item com registro do fato e comunicação à docente.
R2	Inconsistência ou redundância entre itens das duas seções.	Média / alto	Itens sem remissão aos itens relacionados; requisito sem linha na matriz de rastreabilidade.	Revisão cruzada entre seções; revisão de integração síncrona antes da entrega.
R3	Uso de IA sem citação ou sem verificação do conteúdo gerado.	Média / alto	Item sem bloco de autoria/revisão ou referência não conferida no *pull request*.	Nota de uso de IA; conferência obrigatória das referências pelo revisor.
R4	Perda ou sobrescrita de artefatos.	Baixa / alto	Conflitos de integração no repositório.	Versionamento no GitHub, ramos por item e proteção do ramo principal.
R5	Indisponibilidade de integrante por compromissos externos.	Média / médio	Aviso prévio no Discord; ausência em atividade síncrona.	Folga de um dia no cronograma; reprogramação das tarefas assíncronas e priorização dos itens dos quais outros dependem.

Fonte: elaborada pelos autores.